



清华大学计算机科学与技术系本科专业主修课

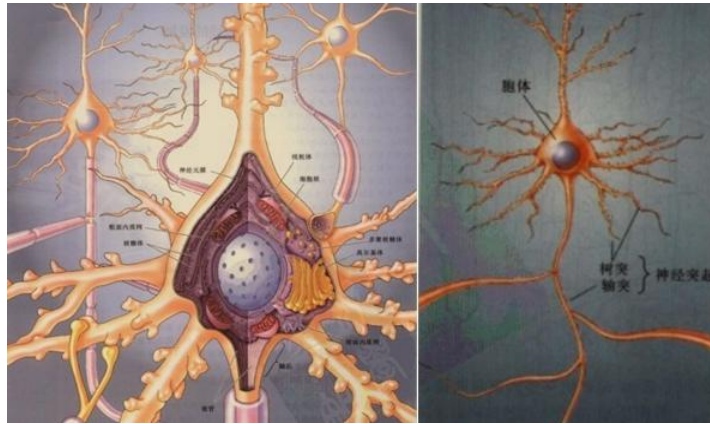
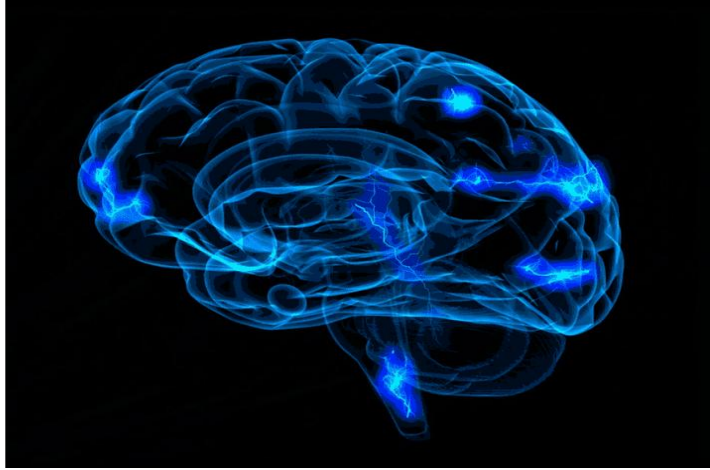
计算机系统结构

第1章 计算机系统结构的基础知识

2026年春季学期

我们为什么需要计算机

第 2 页



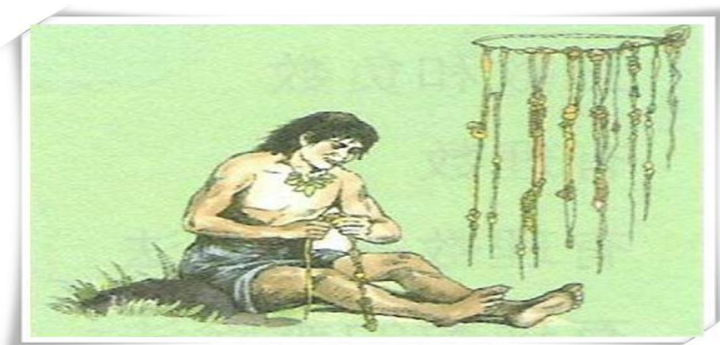
人类大脑约**1000亿**神经元,
100万亿个突触



——节取自江苏台最强大脑节目

什么是计算机

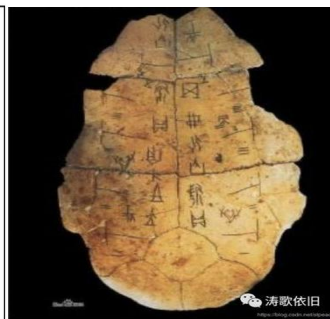
第 3 页



结绳计数（远古时代）

殷甲 骨文	一	二	三	四	五	六	七	八	九	十	百	千	万
	1	2	3	4	5	6	7	8	9	10	100	1000	10000
周秦 金文	一	二	三	四	五	六	七	八	九	十	百	千	万
	1	2	3	4	5	6	7	8	9	10	100	1000	10000
算筹	一	二	三	四	五	六	七	八	九	十	百	千	万
	1	2	3	4	5	6	7	8	9	10	100	1000	10000

刻痕计数（人类最早的数学活动）

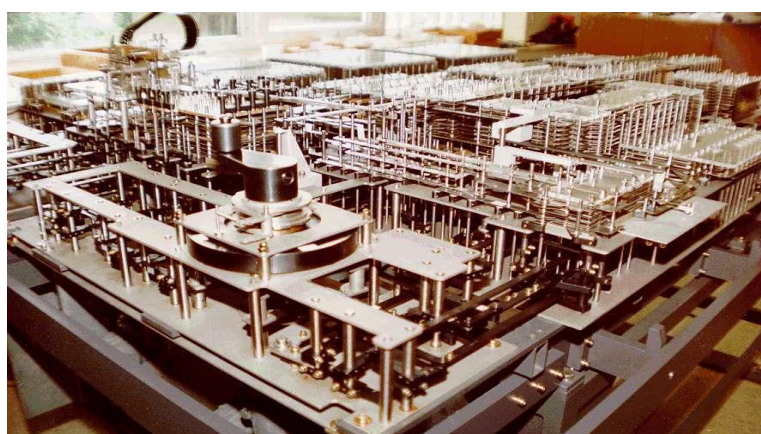


算盘手动计算（~2600年，15世纪）

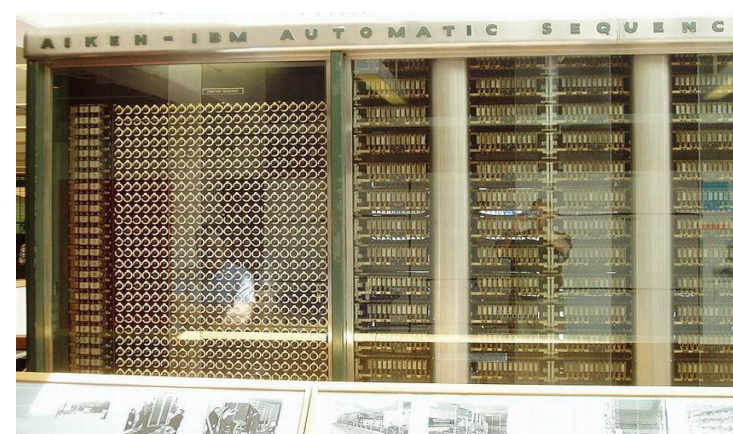
人类的思想变成代码实现 ← 人为地通过信息输入实现信息的处理和输出



契克卡德计算钟（机械，~1624年）



祖斯Z1计算机（机电，1938年）

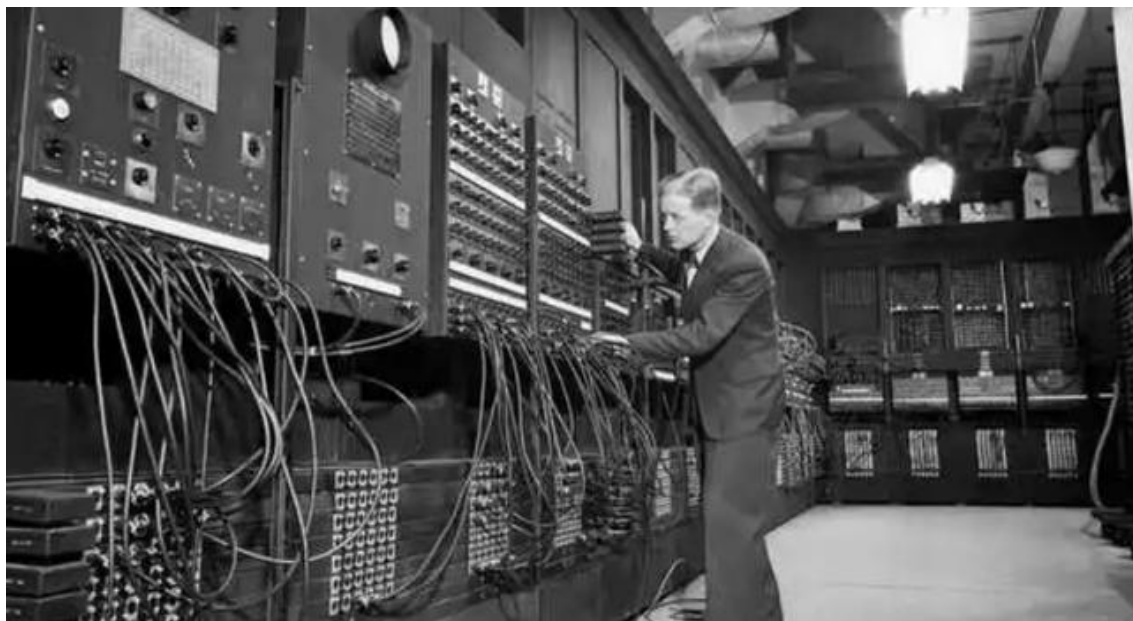


MARK I 计算机（机电，1944年）

第一台通用电子计算机诞生（1946年）

第 4 页

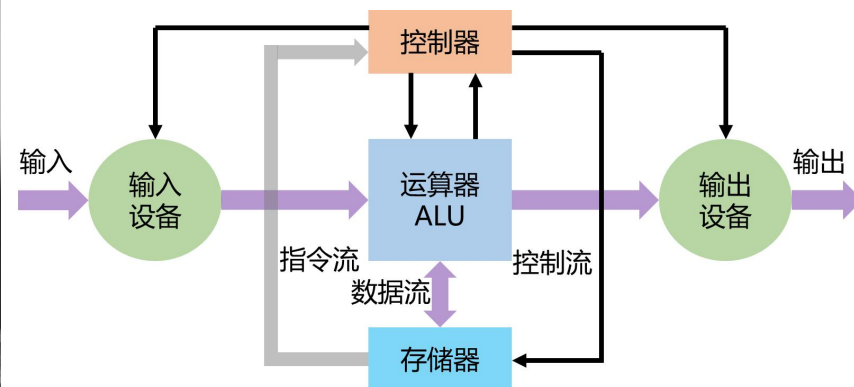
世界上第一台通用计算机是1946年在美国宾夕法尼亚大学诞生的电子数值积分计算机，简称ENIAC（埃尼阿克）



使用18000个电子管，总体积约90立方米，耗电量为140千瓦，运算速度为每秒5000次加法，或者400次乘法，比机械式的继电器计算机快1000倍，美国国防部用这台机器来完成弹道计算。



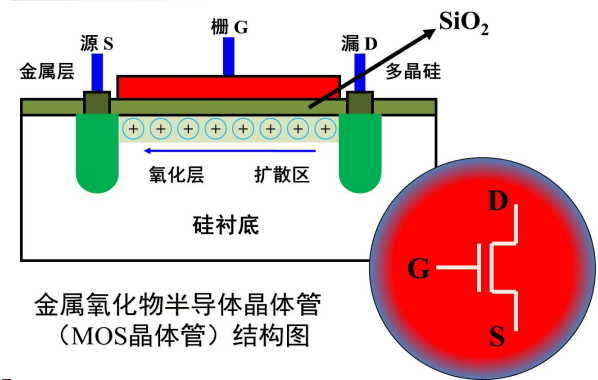
冯·诺依曼
John von Neuman



- 将计算过程描述为由许多条指令按一定顺序组成的程序，并放入存储器保存
- 指令按其在存储器中存放的顺序执行；

- 体积巨大：长50英尺，宽30英尺，重达30吨
- 耗电巨大：每次开机时整个费城西区电灯都黯然失色
- 可靠性差：平均15分钟烧掉一支电子管

第一个晶体管诞生 (1947年)



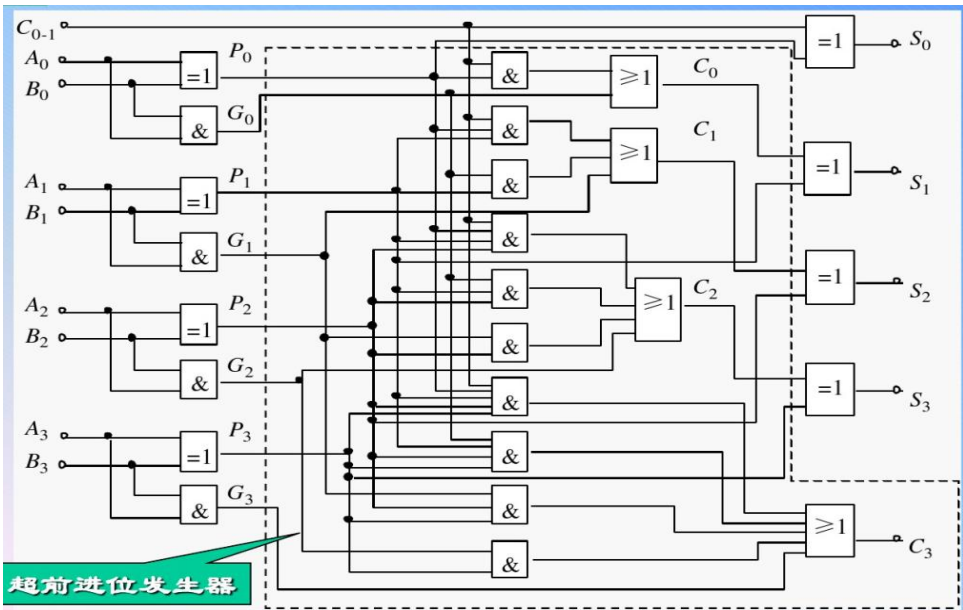
$$8 + 7 = 15$$



$$\begin{array}{r} 1000 \\ + 0111 \\ \hline 1111 \end{array}$$



1947年，巴丁、肖克利和布莱坦发明了第一个晶体管 (1956年诺贝尔奖)



数值计算用逻辑方法实现



TX-0 (MIT)

1953年，晶体管计算机不仅能实现电子管的功能，又具有尺寸小、重量轻、寿命长、效率高、发热少、功耗低等优点。

第一块集成电路的诞生（1958年）

第 6 页



发明人：Kilby
2000年获得诺贝尔物理学奖

集成电路

□ Integrated Circuit, 缩写IC

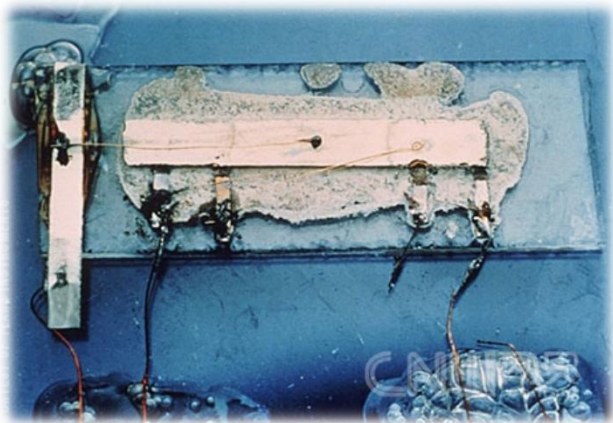
通过一系统特定的加工工艺，将晶体管、二极管等有源器件和电阻、电容等无源器件，按照一定的电路互连，“集成”在一块半导体单晶片（如硅或砷化镓）上，封装在一个外壳内，执行特定电路或系统功能。



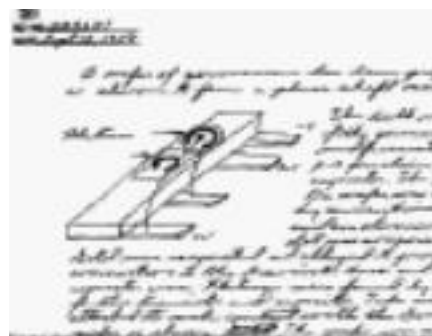
R.Noyce

集成电路之父，仙童半导体公司和英特尔创始人

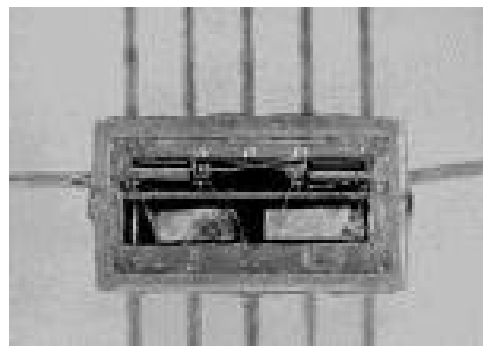
1959年，美国仙童公司的诺伊斯开发出IC平面工艺技术



世界上第一块集成电路
(1958年)



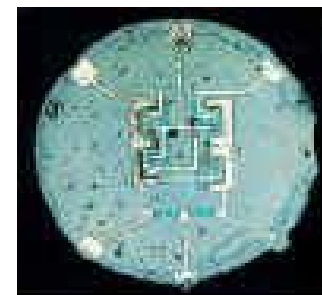
集成电路草图



世界上第一块集成电路

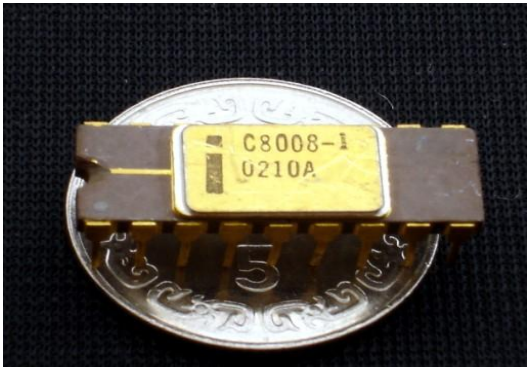


1959年制造的
的IC



1961年制造的
的IC

处理器的诞生与计算机的发展



英特尔 8008 微处理器



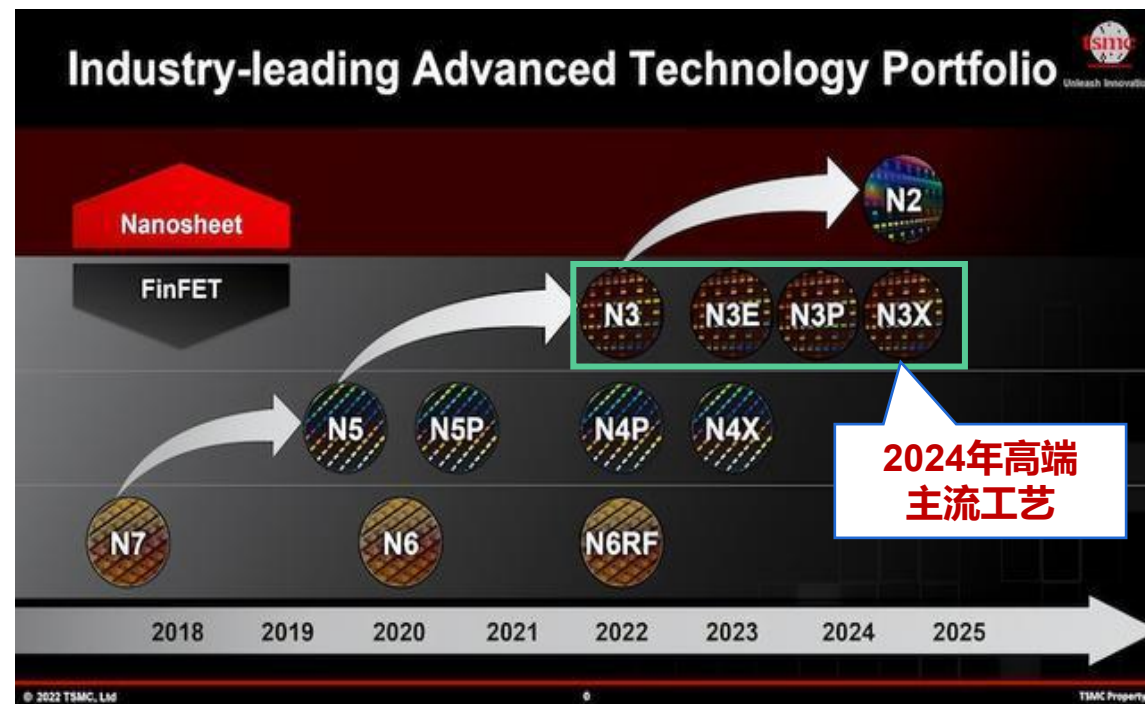
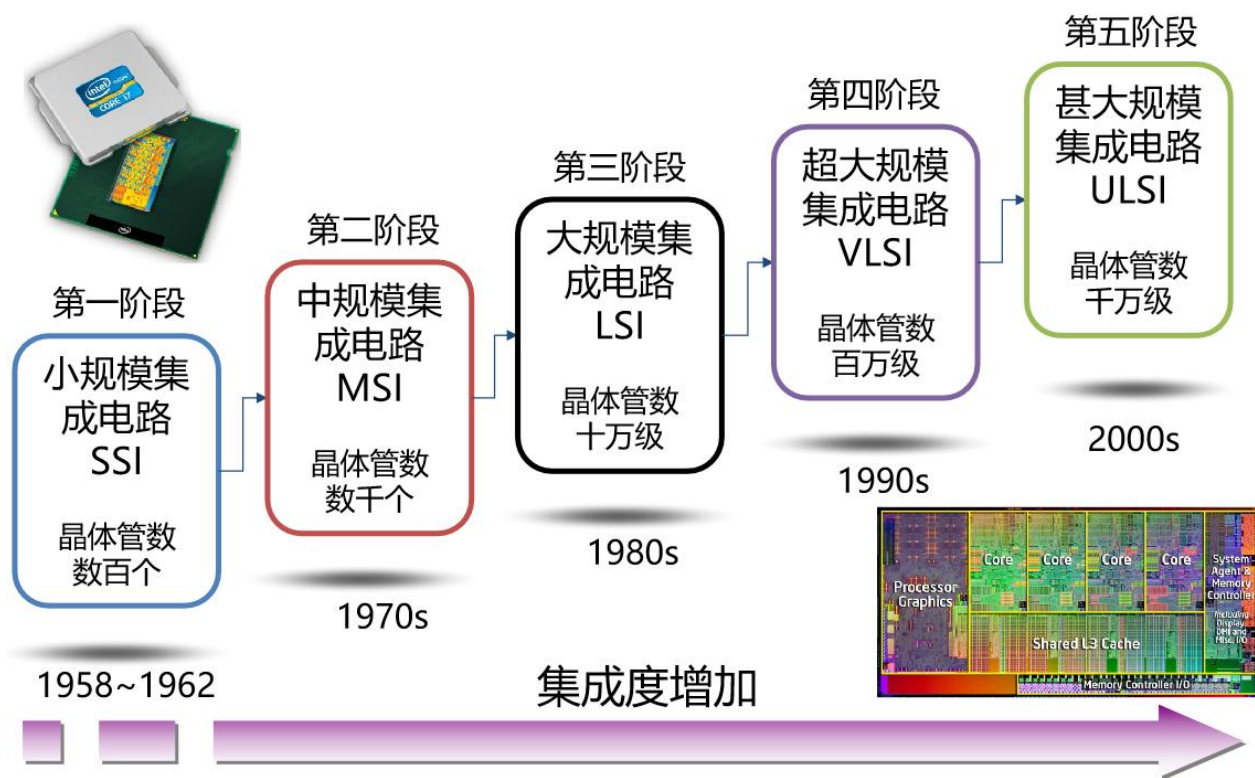
Mark-8 计算机
(第一台个人计算机)

1970s	 Intel® 4004 Processor 问世 1971	 Intel® 8008 Processor 问世 1972	 Intel® 8080 Processor 问世 1974	 Intel® 8086 Processor 问世 1978	 Intel® 8088 Processor 问世 1979	
1980s	 Intel® 286 Processor 问世 1982	 Intel® 386 Processor 问世 1985	 Intel® 486 Processor 问世 1989	2250个晶体管，集成电路工艺10微米，处理4位的数据，每秒运算6万次，频率为108KHz		
1990s	 Intel® Pentium® Processor 问世 1993	 Intel® Pentium® Pro Processor 问世 1995	 Intel® Pentium® II Processor 问世 1997	 Intel® Celeron® Processor 问世 1998	 Intel® Pentium® III Processor 问世 1999	
2000s	 Intel® Pentium® 4 Processor 问世 2000	 Intel® Itanium® Processor 问世 2001	 Intel® Xeon® Processor 问世 2001	 Intel® Itanium® 2 Processor 问世 2003	 Intel® Dual-Core Itanium® 2 Processor 问世 2006	 Intel® Core™2 Duo Processor 问世 2006

计算机制造技术推动计算机发展

第 8 页

摩尔定律（计算机第一定律，1965年）：当价格不变时，集成电路上可容纳的晶体管数目，约每隔18个月便会增加一倍，性能也将提升一倍。

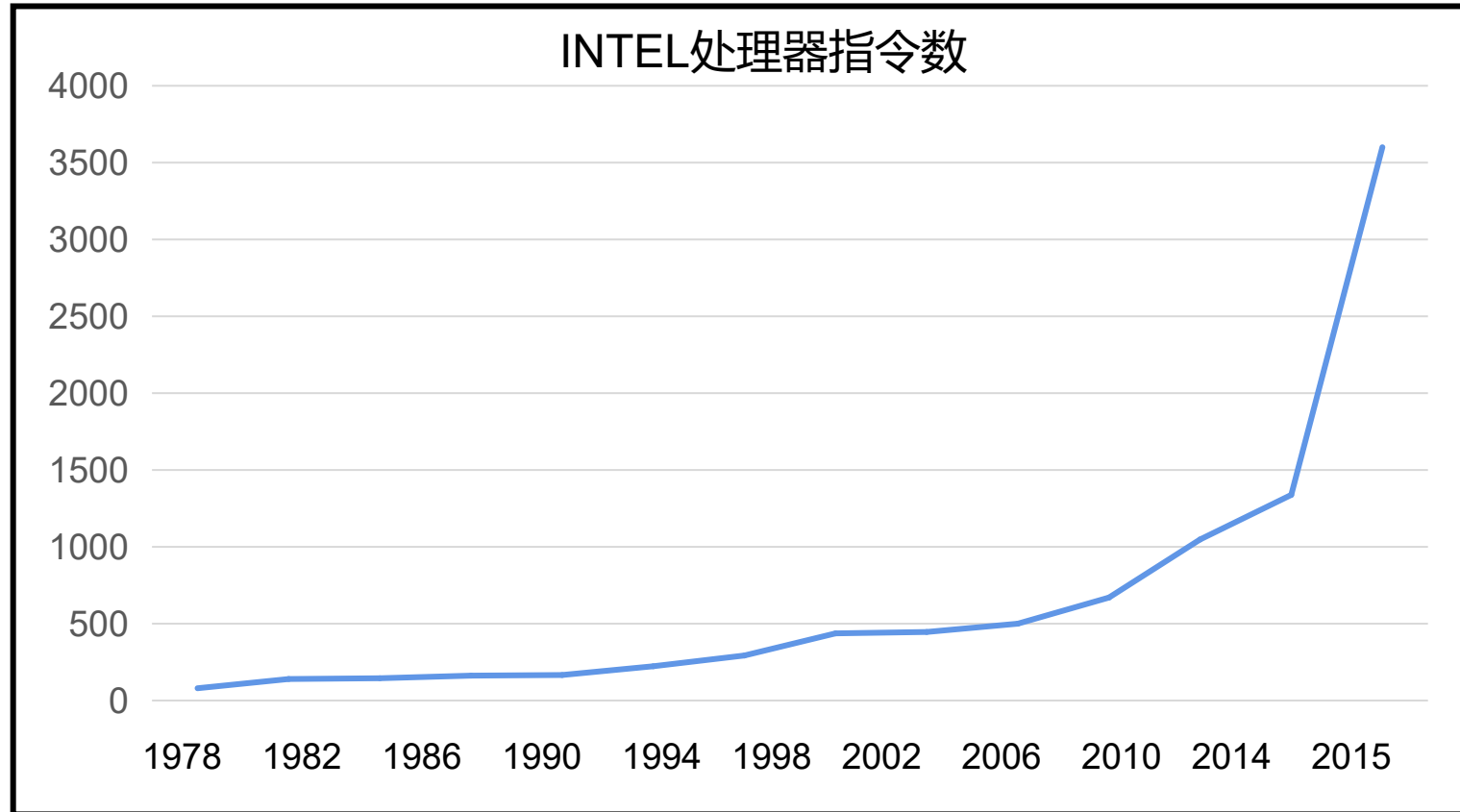


——引自TSMC的发展规划

结论： 归功于以集成电路为代表的计算机制造技术的发展，大规模集成电路和微处理器的出现使得计算机性能每年提升~35%

指令数变得越来越多

第 9 页



X86在1978年诞生时有80条指令，到2015年增长了16倍，到了**1338条**指令，并且仍在增长。

英特尔首席律师史蒂芬·罗杰斯(Stephen Rodgers)和实验室主管理查德·A·乌利希(Richard A. Uhlig)撰写博客在英特尔的博客上有着**3600条**指令的统计结果，这意味着X86指令的增长速率提高到了（在1978年到2015年之内）每四天增长一条。

单个指令的功能变得越来越强大

第 10 页

Intel处理器指令集主要包括X86、EM64T、MMX、SSE、SSE2、SSE3、SSSE3 (Super SSE3)、SSE4A、SSE4.1、SSE4.2、AVX、AVX2、AVX-512、VMX等指令集

INTEL处理器的 SSE4.2 指令集

`_mm_pcmpistrm(_m128 a, _m128 b)`

一次对一组16个字符与另一组字符作比较，也就是说一个指令可以作最多 $16 \times 16 = 256$ 次比较

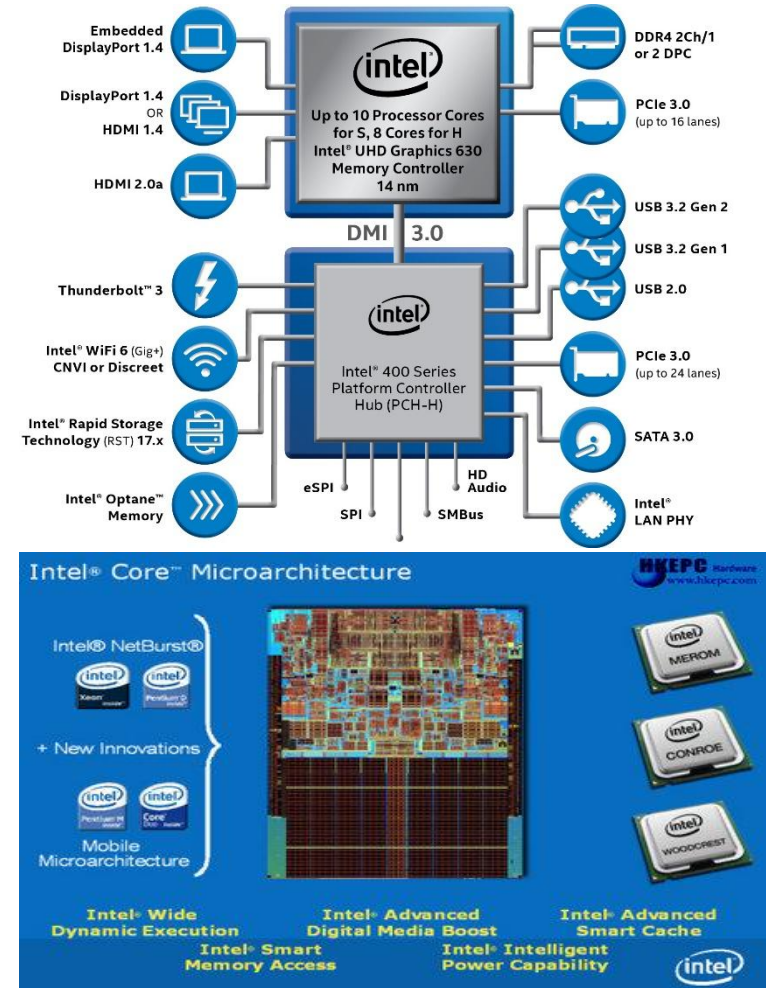
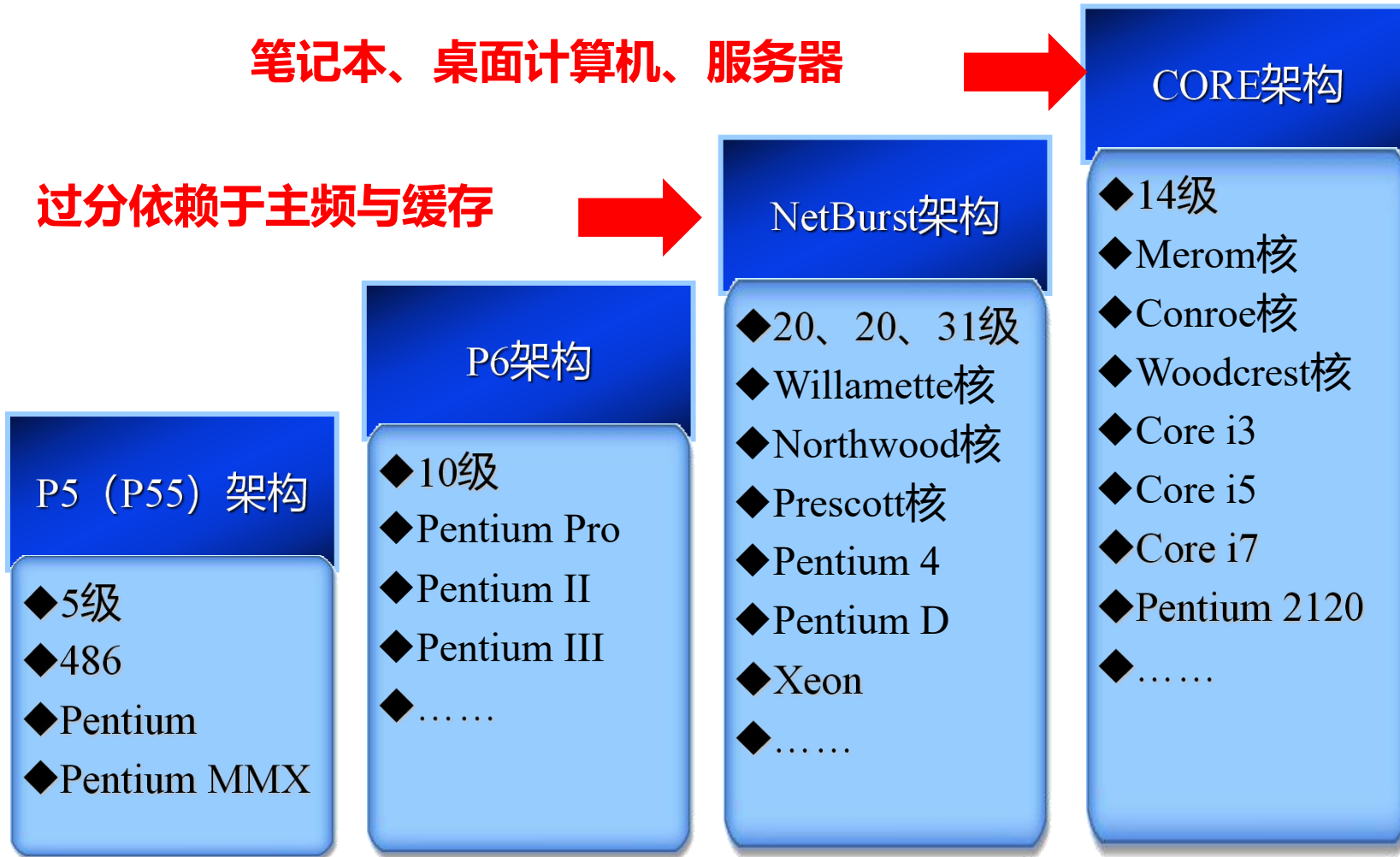
SSE2指令集（数据流单指令多数据扩展指令集2），新增了144个指令，扩展了MMX技术和SSE技术媒体应用，还有助于加速内容创建、财务、工程和科学应用

流水线变得越来越复杂

第 11 页

笔记本、桌面计算机、服务器

过分依赖于主频与缓存



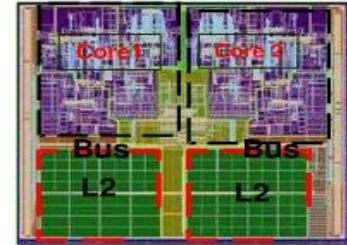
进入多核与众核处理器

第 12 页

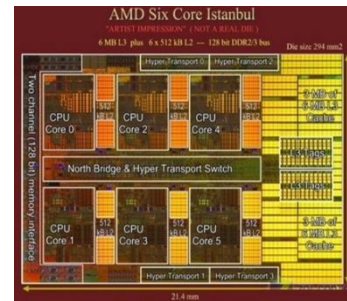


英特尔处理器实验室主任
Fred Pollack

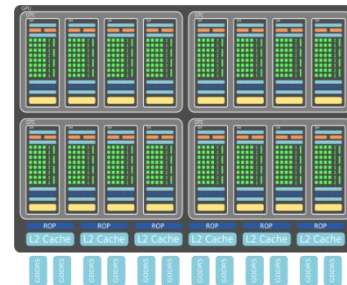
- 从386起，英特尔每一新架构需要两到三倍的晶片面积，而性能只提升1.4到1.7倍。
- 简言之，性能的提升与复杂性的平方根成比例。两代处理器，性能每提升一倍，复杂性便增加4倍。
- 如一个处理器的硬件逻辑提高一倍，至多能提高性能40%。采用两个简单处理器构成一个相同硬件规模的双核处理器，可获得70% ~ 80%的性能提升。



双核处理器



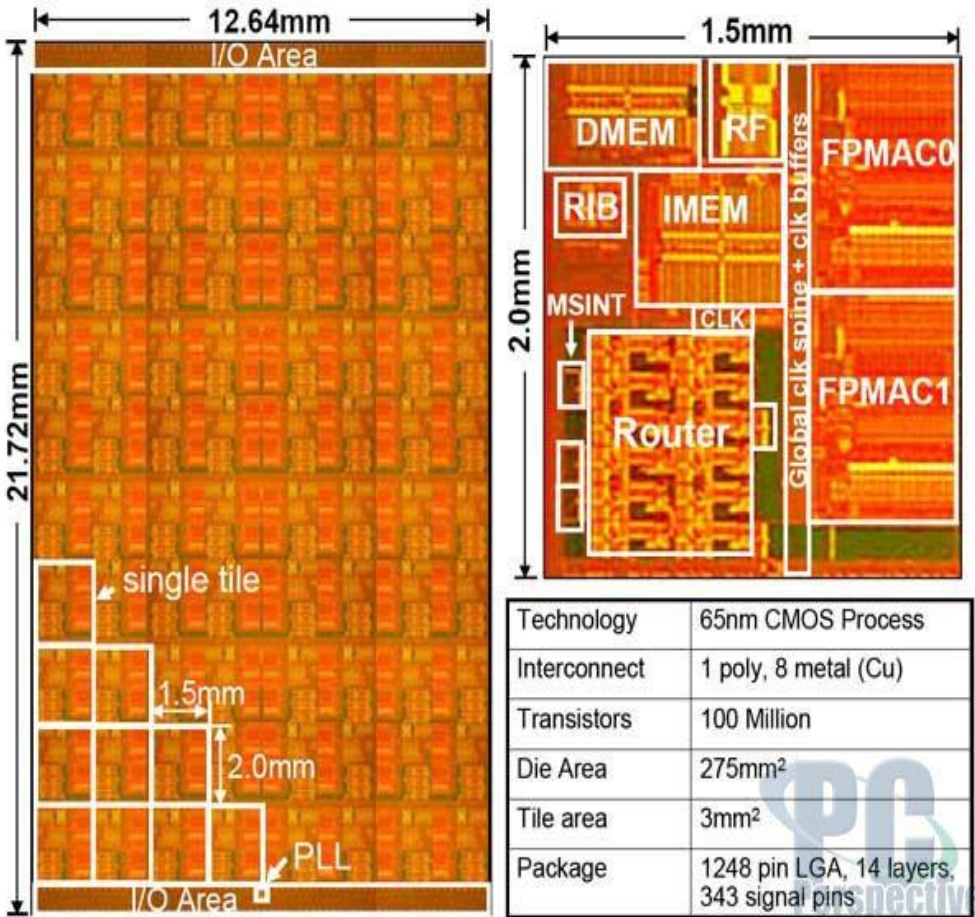
多核处理器



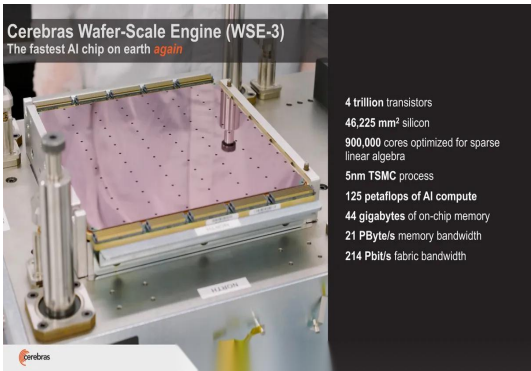
众核处理器

2004年，Intel公司取消了高性能单一处理器的研究计划！

Terascale 80核处理器（Polaris）



WSE-2: 专门面向人工智能任务，拥有2.6万亿个晶体管，**85万核心** (2021年)



WSE-3: 拥有的晶体管数量达到了4万亿个，拥有**90万个AI核心**，可用于训练业内一些最大的人工智能模型 (2024年)

Cerebras Wafer Scale Engine
有史以来最大的芯片

AI的发展要素

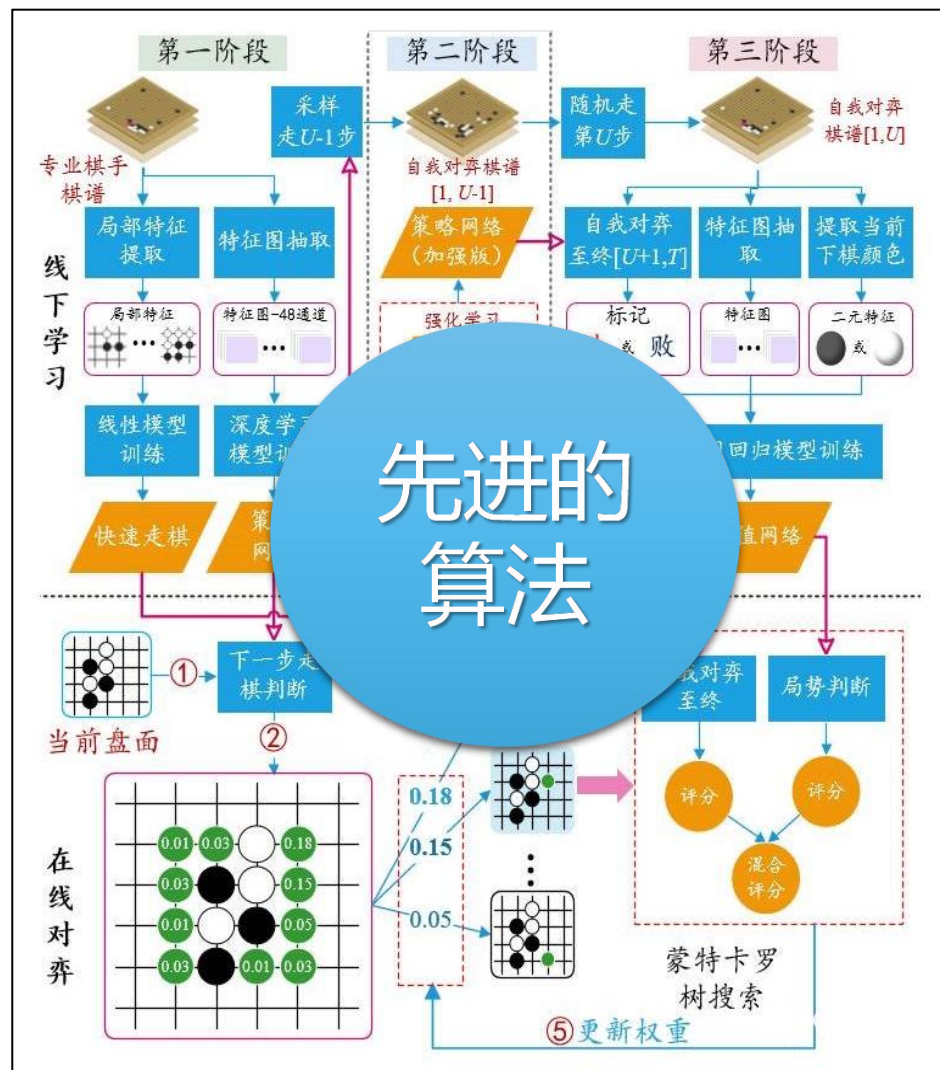
第 14 页



李世石对弈AlphaGo
AlphaGo-Lee: 1202个CPU, 176个GPU



柯杰对弈AlphaGo
AlphaGo Master: 1920个CPU, 280个GPU



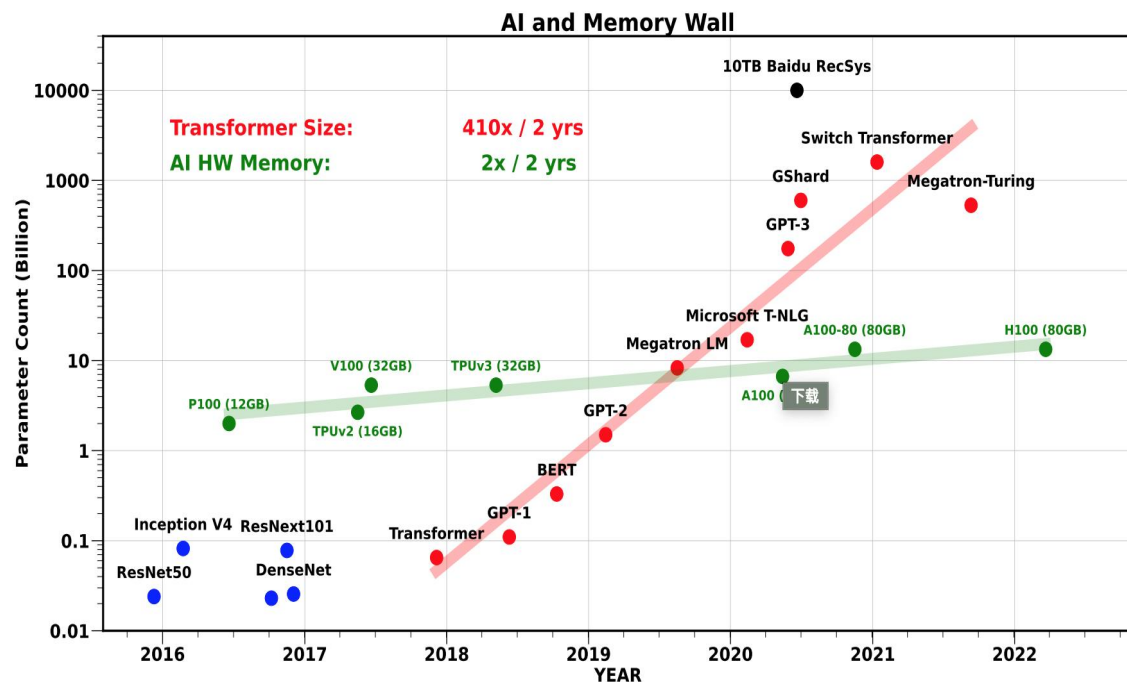
强大的
计算能力



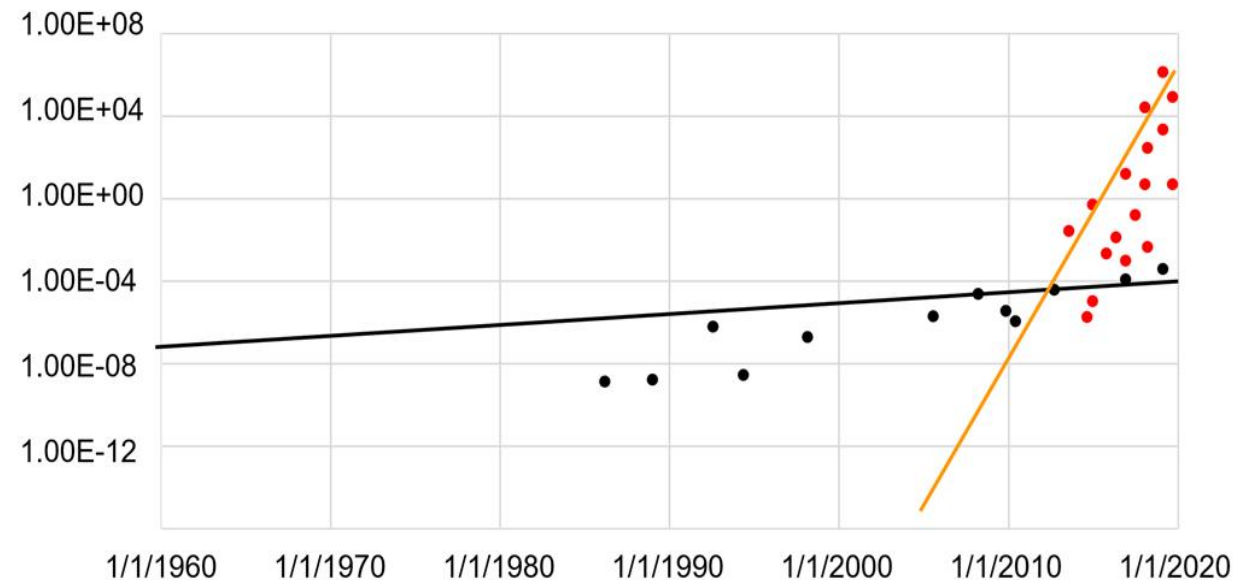
海量
数据

AI需求促进新型计算机架构创新

第 15 页



AI模型所需参数量与存储器数据处理能力的对比



AI发展推动世界已经进入算力时代, 过去10年计算性能增长100万倍, 预计未来10年计算性能需求增长100万倍

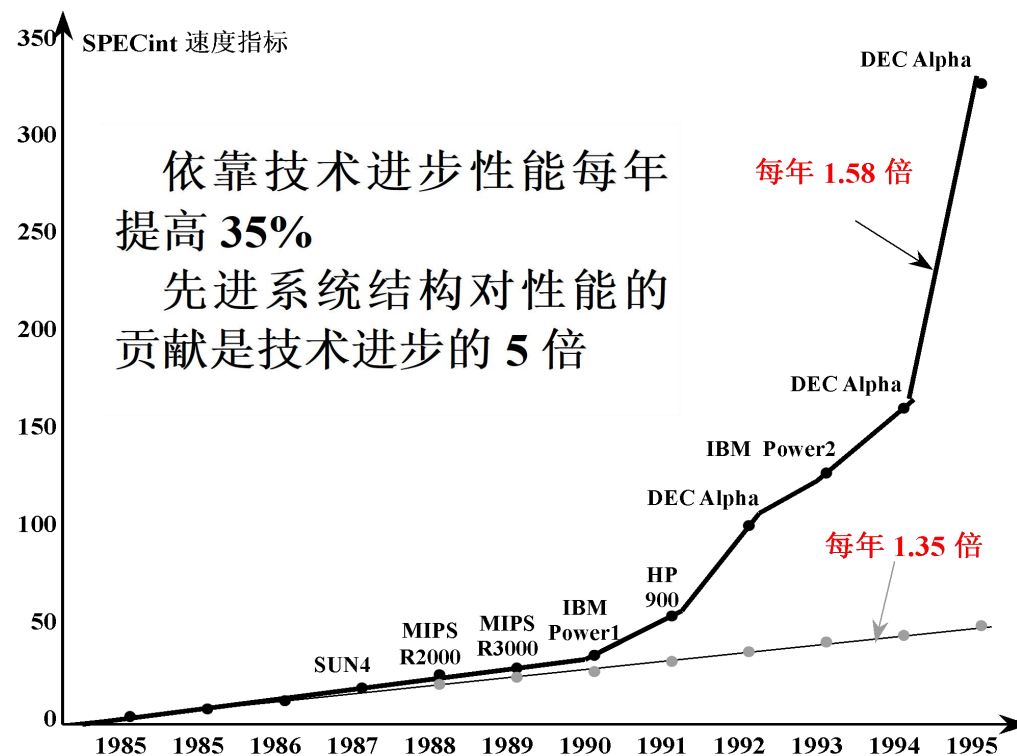
计算机技术发展总结

第 16 页

自第一台通用电子计算机诞生以来的七十多年中，计算机技术一直在以惊人的速度发展。对于许多应用来说，现在的高性能处理器的性能已经超过了十几年前的超级计算机，这主要受益于：

关键技术一：以超大规模集成电路（VLSI）为代表的计算机制造技术的发展

关键技术二：以RISC架构和并行处理机为代表计算机系统结构的创新，
→ 授课内容



计算机性能每年提升50%

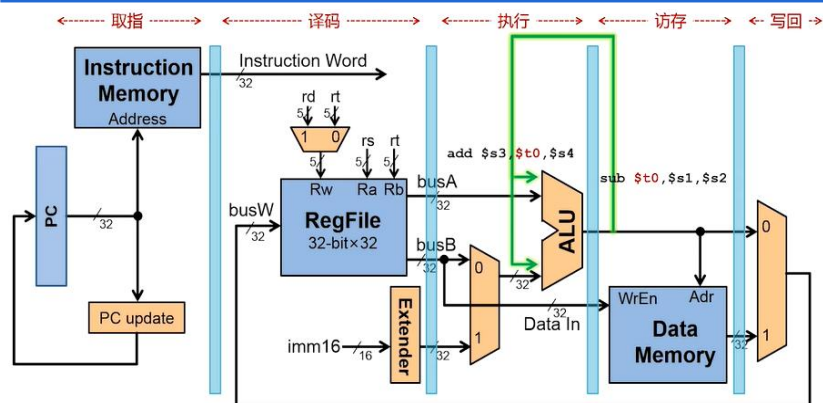
继续学习什么

第 17 页



计算机组成原理

- 为什么增加一条指令？增加哪一条指令？
- 如何设计复杂流水线？如何控制？
- 如何设计多核？如何实现多核互连？
- 为什么设计多级CACHE？原理？大小？管理？
- 还有什么创新？



计算机系统结构：
知其然，知其所以然！

授课团队与参考教材

第 18 页

■主讲教师

李兆麟 教授 lzl73@mail.tsinghua.edu.cn 办公室：自强楼1-512室

■助教团队

崔轶锴 博士生 cyk23@mails.tsinghua.edu.cn 15910826331

李唯可 博士生 wk-li20@mails.tsinghua.edu.cn 15338830391

■主要参考教材

计算机系统结构 张晨曦、王志英等，清华大学出版社，2021年（主要教材）

计算机系统结构 郑纬民、汤志忠等，清华大学出版社，2003年（参考教材）

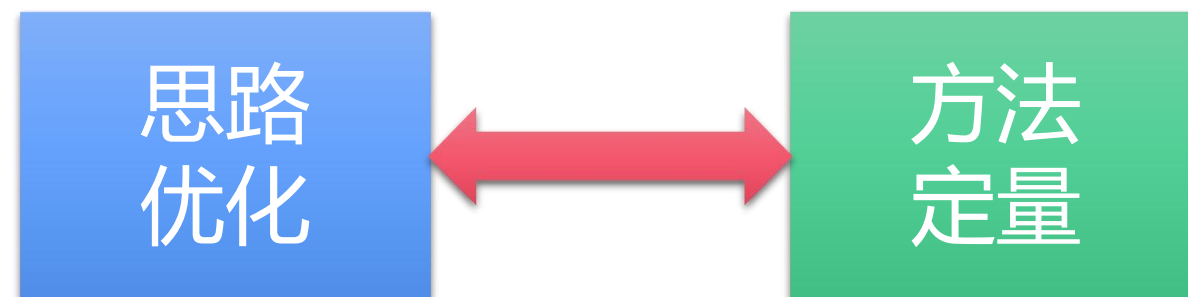
计算机系统结构学习指导与题解 张晨曦等，清华大学出版社，2018年（作业参考教材）

共十章（15讲），具体包括：

- ① 基本概念&性能评价
- ② 指令系统设计
- ③ 流水线技术基础
- ④ 向量处理
- ⑤ 指令级并行-硬件方法
- ⑥ 指令级并行-软件方法
- ⑦ 存储系统
- ⑧ 互连网络
- ⑨ 多处理机
- ⑩ 先进架构

□ **教**：按照层次结构来组织教学内容，采用概念自顶向下、从整体到局部地逐层展开的教学方式；

□ **学**：强调学生从整体、系统的角度来看问题，既要有系统的观点，把握全局，又要有层次的观点，分而治之。



■学分：3分

■学时：48学时授课（每周四上午第二大节）

➤注意调课安排（**4月30日**课程停上）

■考试：课程出勤(10%)+课程作业(10%)+大作业（20%）+期末考试(60%)

➤课程出勤(10%)，第一节或第二节后，共15次，考前确认（**真名+学号**）

➤课程作业（10%），每两章复习一次，留作业一次，**设置截止时间，需按时间提交**

➤大作业（20%）：第5讲指令调度（10分），第7讲CACHE系统（10分）

➤期末考试（60%）：17-18周（统一安排考试）（6月15-19日*）

➤**考前安排一次补交：课程作业(5%)+大作业（10%）**

■ 判断对错题（15分）

① 《计算机系统结构》是我最喜欢的课程之一。（√）

■ 单选题（15分）

② 我最喜欢的学习环节包括（ A或B或C或D ）

A：雨课堂考勤 B：课堂作业日常复习 C：大作业动手能力锻炼 D：期末考核

■ 填空与计算题（30分）

③ 《计算机系统结构》的学习占用我的学习时间比例是（ 20% ）

■ 综合分析题（40分）

④ 请分析学好《计算机系统结构》需要哪些因素？

学好 = A（认真负责） + B（学科热爱） + C（合理规划） + D（共同努力）

答疑与反馈

第 22 页

■答疑：集中在周四下午2点-5点（自强楼1-512室或5层楼助教）

- 授课内容补充（基本点、难点）
- 助教集中答疑（重点习题、大作业）
- 日常补充答疑（助教微信群+网络学堂）

■反馈：第8、12周（拟定周四下午集中+平时随时反馈）

■如果有好的建议与意见，随时欢迎及时反馈助教！

■有志于从事高性能计算架构与编译的同学请关注！

- 参与修改PPT与教学内容；参与课题组科研实践



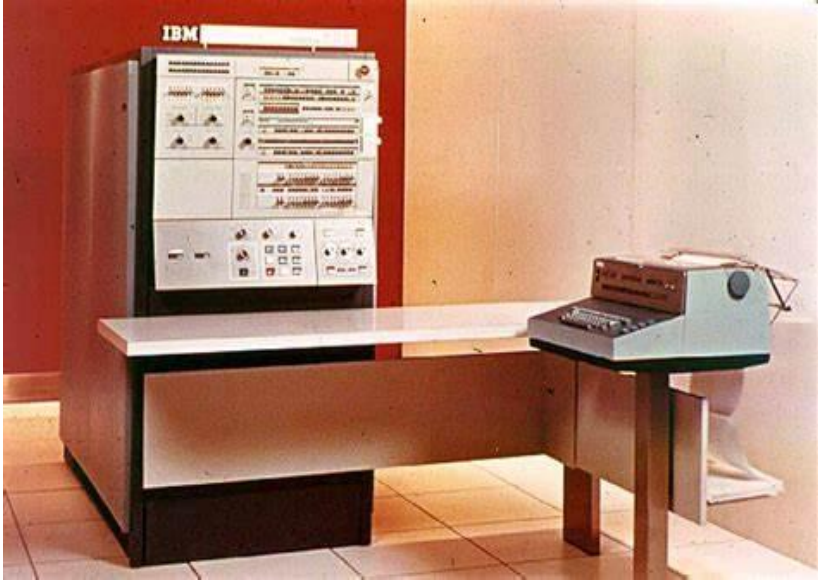
**充分发挥
助教的作用**

- 1.1 [计算机系统结构的基本概念](#)
- 1.2 [计算机系统的设计](#)
- 1.3 [计算机系统的性能评测](#)
- 1.4 [计算机系统结构的发展](#)
- 1.5 [计算机系统结构中并行性的发展](#)

1.1 计算机系统结构的基本概念

1.1.1 计算机系统结构的基本概念

第 25 页



IBM 360计算机

1964年，吉恩·阿姆达尔（Gene Amdahl）担任主设计师，历时四年研发成功，标志着第三代计算机——集成电路计算机诞生，是世界上首个指令集可兼容计算机。

Computer Architecture

传统定义：

- ◆ Amdahl 于1964年提出（IBM 360）
- ◆ （机器语言）程序员所看到的计算机的属性，即概念性结构与功能特性。
- ◆ 计算机系统软硬件的交界面

广义定义：

- 指令集结构（Instruction Set Architecture）
- + 计算机组织（Computer Organization）
- + 计算机实现（Computer Implementation）

1.1.1 计算机系统结构的基本概念

第 26 页

- 体系结构比较重要
- 从系统这一级来看计算机
 - ◆ 计算机专业人的标签
 - ◆ 未来的计算机总工程师
- 建立整机系统和处理器概念
- 总体设计（设计策略）
- 计算机和处理器的缔造者，而非简单的使用者



架构给人
以简洁美、
线条美、
和层次美

1.1.2 计算机系统的层次结构

第 27 页

本质：人类的思想变成代码实现 ← 人为地通过信息输入实现信息的处理和输出



科学计算



自动驾驶



机器人

如何研究计算系统 → 以应用为导向

1.1.2 计算机系统的层次结构

第 28 页

■从计算机语言的角度，把计算机系统按功能划分成多级层次结构

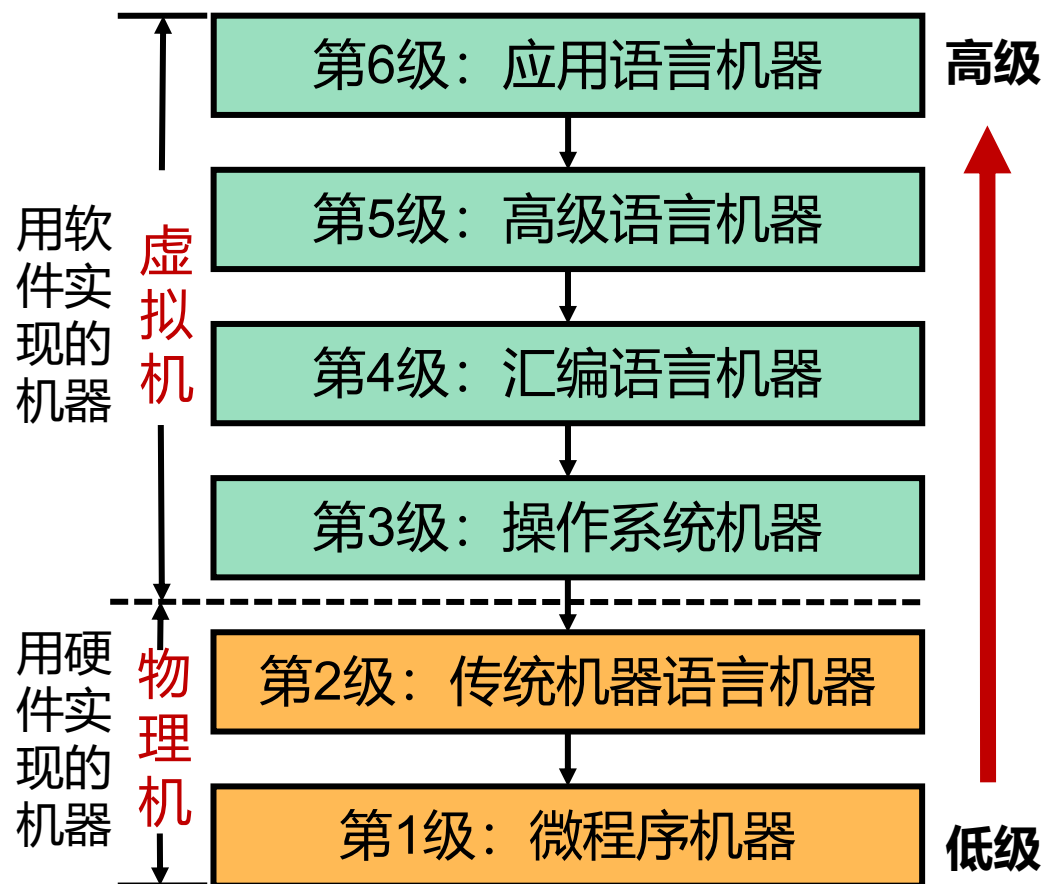


■计算机系统由密切相关的硬件与软件组成

1.1.2 计算机系统的层次结构

第 29 页

■ 每一层以一种计算机语言为特征，计算机语言从低级向高级发展



高一级语言的语句相对于低一级语言功能更强，更便于应用，但又都以低级语言为基础，因此需进一步明确4个问题：

- 1、高一级机器上的程序如何在低一级机器上执行？
- 2、传统机器语言与微程序的区别？
- 3、操作系统机器的执行机制？
- 4、虚拟机是否一定由软件实现？

1.1.2 计算机系统的层次结构

第 30 页

问题1：高一级机器上的程序如何在低一级机器上执行？

■ 高级→低级：**翻译与解释**，或两者的结合

➤ **翻译**：先用转换程序把高一级机器上的程序转换为低一级机器上等效的程序，然后再在这低一级机器上运行，实现程序的功能。

➤ **解释**：对于高一级机器上的程序中的每一条语句或指令，都是转去执行低一级机器上的一段等效程序。执行完后，再去高一级机器取下一条语句或指令，再进行解释执行，如此反复，直到解释执行完整个程序。

➤ 上面3级：经常用翻译；下面3级：经常用解释

一个简单的高级语言程序

```
C = A * B ;    // 乘法操作  
E = C + D ;    // 加法操作
```

翻译执行结果：

```
.....  
mac R0, R1, R2, R3 ; // 乘加指令  
.....
```

解释执行结果：

```
.....  
mul R0, R1, R2 ;    // 乘法指令  
.....  
add R2, R3, R4 ;    // 加法指令  
.....
```

翻译执行：运行效率高，但实现困难、调试复杂； 解释执行：灵活，但整体优化困难、运行效率低

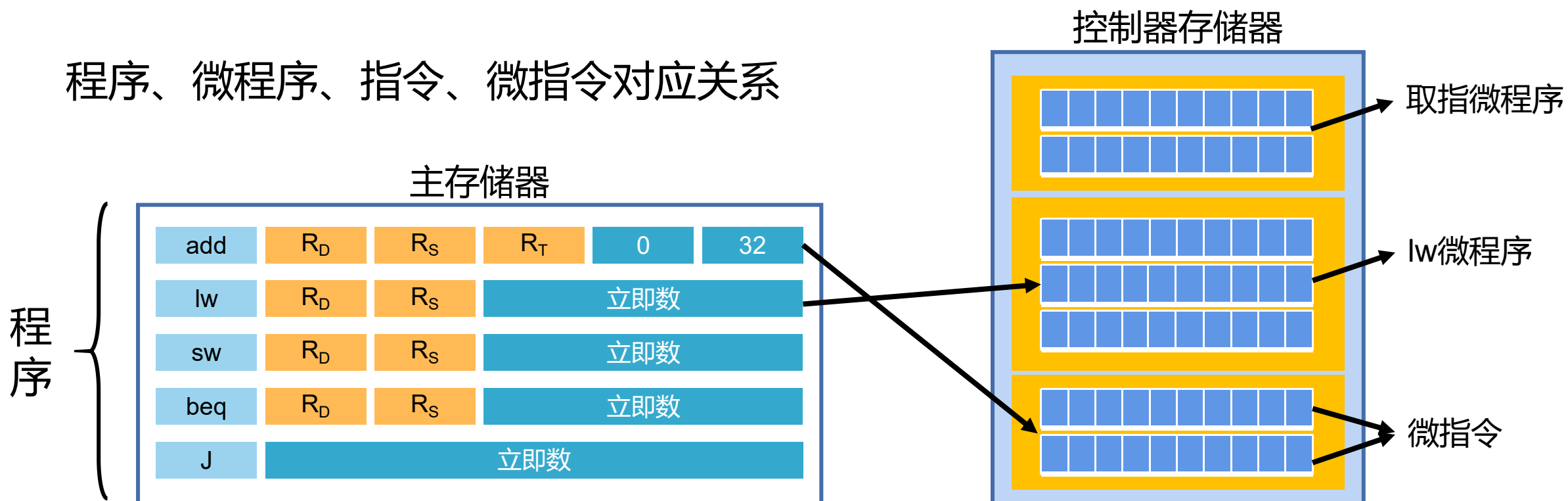
1.1.2 计算机系统的层次结构

第 31 页

问题2：传统机器语言与微程序的区别？

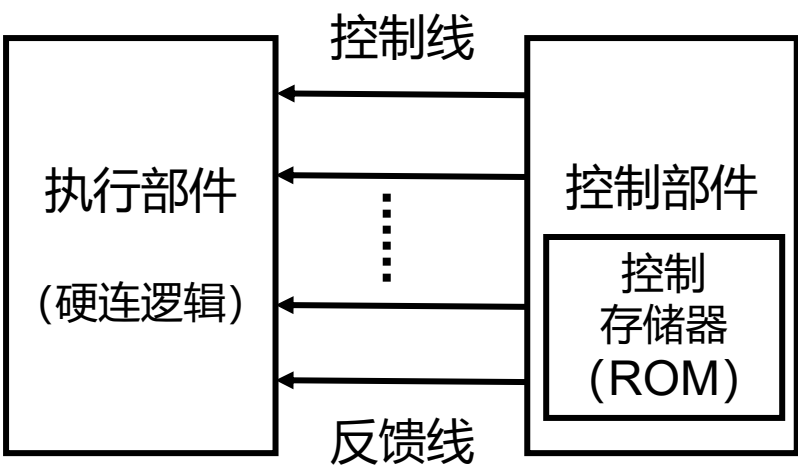
- 微程序(microprogram)是英国剑桥大学教授M.V.Wilkes在1951年提出的
 - 基本思想：将一条机器指令编写成一段包含若干条微指令的微程序，由硬件来解释执行。

程序、微程序、指令、微指令对应关系



1.1.2 计算机系统的层次结构

■ 微程序的执行机制：在微程序机器中，CPU内部有一个控制存储器，用于存放各种机器指令对应的微程序段；当CPU执行机器指令时，会在控制存储器里寻找与该机器指令对应的微程序，取出相应的微指令来控制执行各个微操作，从而完成该程序语句的功能。



■ 微程序设计技术是利用软件技术来实现硬件设计的一门技术，但有的计算机没有采用微程序技术，第2级的指令系统是由硬件直接解释执行，如**RISC架构处理器**。

微程序机器	不采用微程序的机器语言机器
优点： 硬件电路相对简单，指令系统的规整性，灵活性，可维护性比较好； 缺点： 由于增加了读取微指令的时间导致执行速度慢	优点： 硬件执行速度快 缺点： 指令系统相对简单，如果采用复杂指令则需要采用复杂的逻辑电路

1.1.2 计算机系统的层次结构

第 33 页

问题3：操作系统机器的执行机制？

■ 操作系统机器：传统机器级指令+**操作系统级指令**

- 操作系统级指令用于实现操作系统功能的调用，例如打开/关闭文件、读写文件等
- 用这一级语言编写的程序是由第3级和第2级共同执行的，其中只有操作系统级指令是由操作系统解释执行的。

```
int fopen(char *filename, int access)
int fclose(int handle)
int fread(int handle, void *buf, int count)
int fwrite(int handle, void *buf, int count)
```

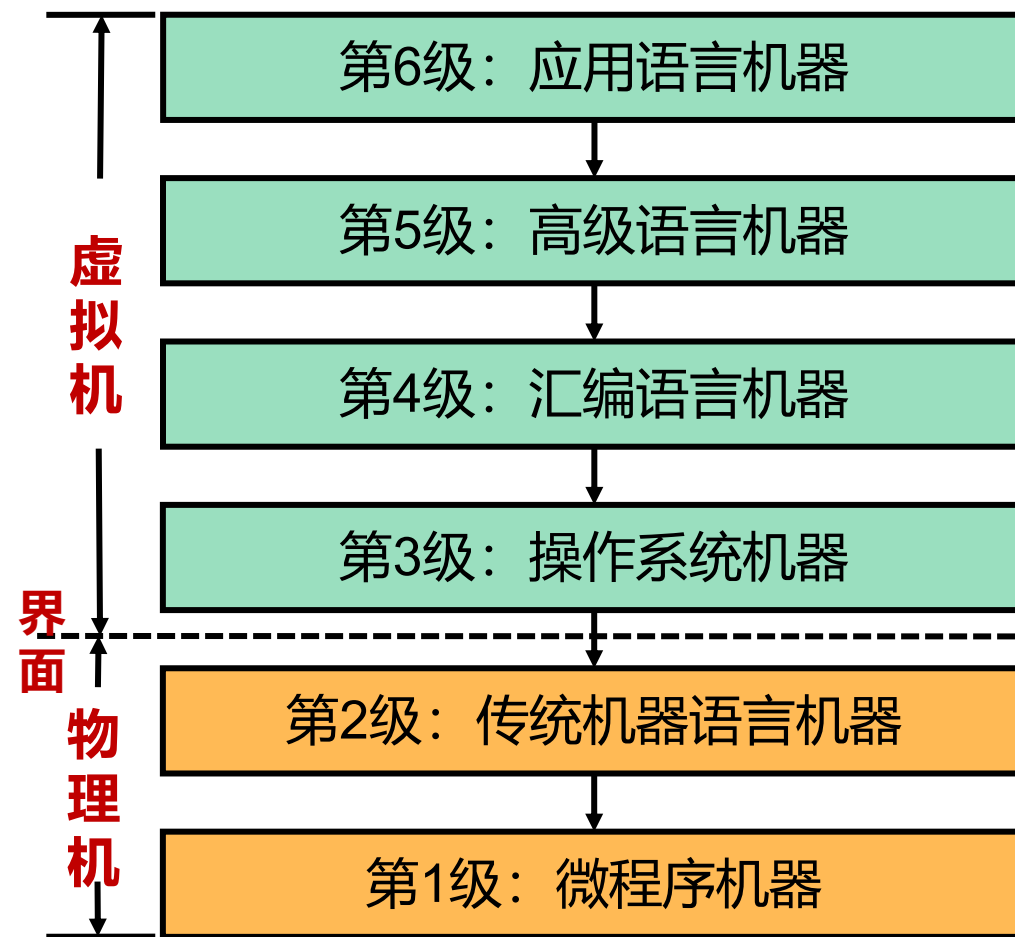
} 由操作系统解释执行

1.1.2 计算机系统的层次结构

第 34 页

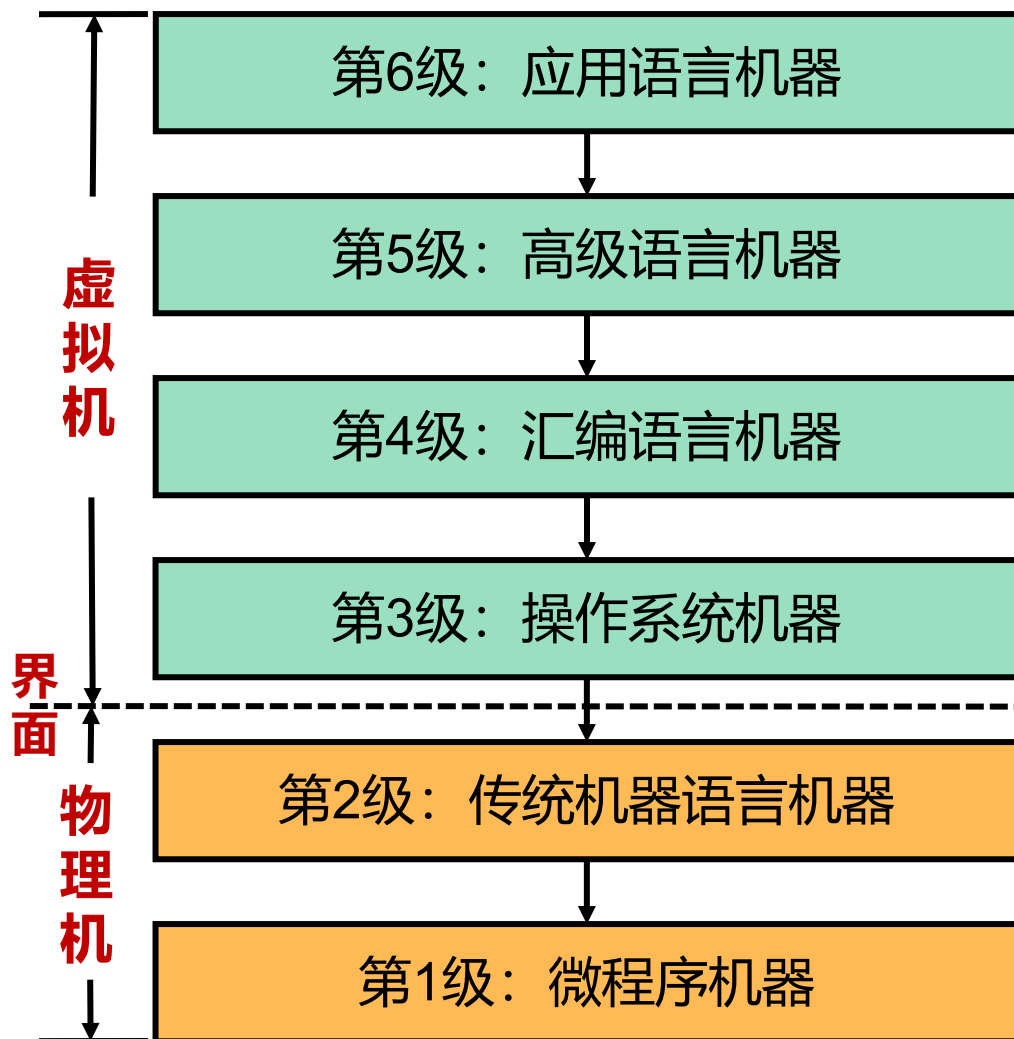
问题4：虚拟机是否一定由软件实现？

- **虚拟机**：用软件实现的机器（Virtual Machine） Vs. 物理机（Physical Machine）
 - **虚拟机**不一定完全由软件实现，有些操作可以由硬件或固件实现
 - **固件（Firmware）**：固化的软件或具有软件功能的硬件，实现方法：把程序写入EEPROM只读存储
 - ✓ 应用：设备内部保存的设备“驱动程序”
 - ✓ 优点：通过修改固件中的软件代码就可以改变其功能，与硬连逻辑相比，灵活性大
 - ✓ 缺点：速度较慢



1.1.2 计算机系统的层次结构

第 35 页



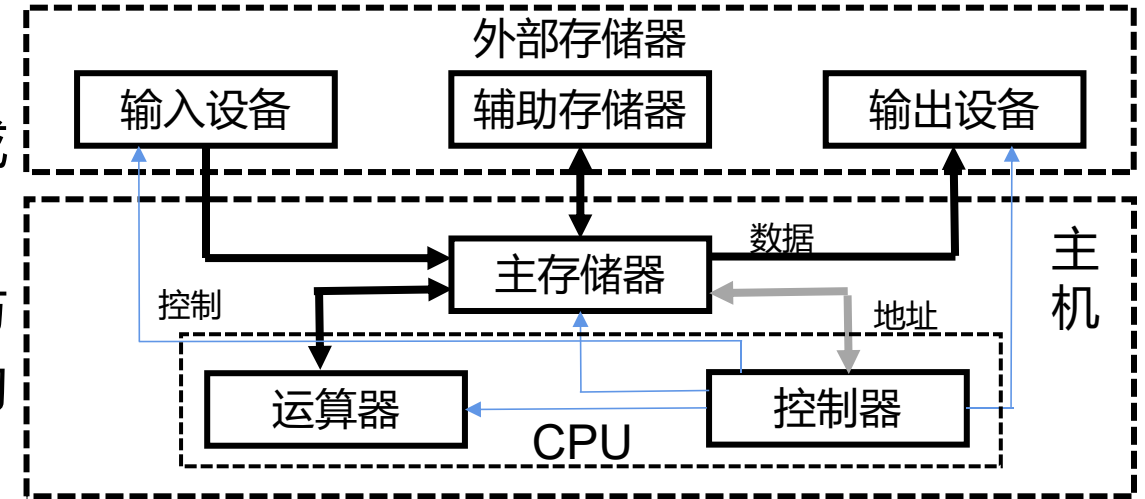
1. **经典定义：**传统机器程序员看到的计算机属性，按照计算机系统的多级层次结构，不同级程序员所看到的计算机具有不同的属性**(透明性)**。
2. **广义定义：**
 - 指令集结构 (Instruction Set Architecture)
 - + 计算机组织 (Organization)
 - + 计算机实现 (Implementation)

1.1.3 计算机组成和计算机实现

第 36 页

■ 计算机组成：计算机系统结构的逻辑实现

- 包含物理机器级中的数据流和控制流的组成以及逻辑设计等。
- **着眼于：**物理机器级内各事件的排序方式与控制方式、各部件的功能以及各部件之间的联系。



计算机组成

■ 计算机实现：计算机组成的物理实现

- 包括处理机、主存等部件的物理结构，器件的集成度和速度，模块、插件、底板的划分与连接，信号传输，电源、冷却及整机装配技术等。
- **着眼于：**器件技术（起主导作用）、微组装技术。



计算机实现

1.1.3 计算机组成和计算机实现

第 37 页

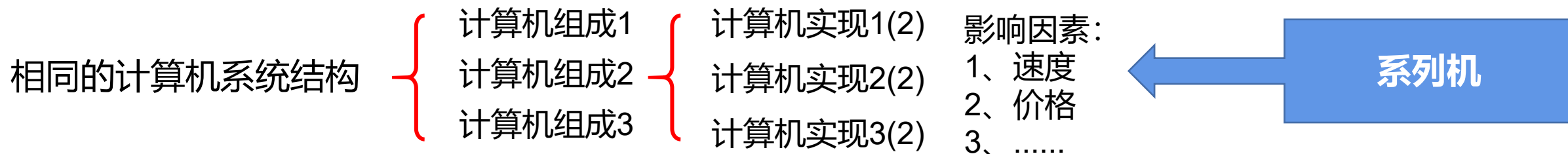
计算机系统结构、计算机组成与计算机实现的区别：

■ 指令系统设计与实现：

- **计算机系统结构**：确定指令系统中**是否有乘法指令**
- **计算机组成**：乘法指令是用**专用的乘法器还是用加法器经过多步操作实现**
- **计算机实现**：乘法器、加法器的**物理实现**，如器件的选定及所用的微组装技术

■ 主存系统设计与实现：

- **计算机系统结构**：确定**主存容量与编址方法**（按位、字节、字或双字访问）
- **计算机组成**：确定**主存速度、逻辑结构**
- **计算机实现**：主存系统的**物理实现**，如器件的选定、逻辑电路的设计、微组装技术等



1.1.3 计算机组成和计算机实现

第 38 页

■ **系列机：**由同一厂家生产的具有相同系统结构、但具有不同组成和实现的一系列不同型号的机器

例如，IBM公司的IBM370系列机有370、115、125、135、145、158、168等从低速到高速的各种型号。它们**具有形同的系统结构，而采用不同的组成和实现技术**，有不同的性能和价格。在低挡机上指令的分析和指令的执行顺序进行，而在高档机上采用重叠、流水和其他并行处理方式。

结论：具有相同系统结构的计算机可以采用不同的计算机组成，同一种计算机组成又可以采用多种不同的计算机实现。



1.1.4 计算机系统结构的分类

第 39 页

常见的计算机系统结构分类法有**3**种：

1. Flynn分类法
2. 冯氏分类法
3. Handler分类法

1.1.4 计算机系统结构的分类

第 40 页

第一种：弗林（Flynn）分类法

- 按照指令流和数据流的多倍性进行分类。
 - ✓ 指令流：计算机执行的指令序列 m
 - ✓ 数据流：由指令流调用的数据序列 n
 - ✓ 多倍性：在系统最受限的部件上同时处于同一执行阶段的指令或数据的最大数目。
- 把计算机系统的结构分为**4**类 • • •

最常用的分类法

1.1.4 计算机系统结构的分类

第 41 页

- 类型一：单指令流单数据流 **SISD**
(Single Instruction stream Single Data stream)

$$m=1, n=1$$

- 类型二：单指令流多数据流 **SIMD**
(Single Instruction stream Multiple Data stream)

$$m=1, n \geq 2$$

IS: 指令流

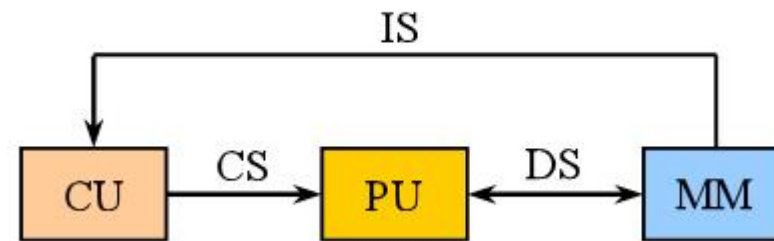
DS: 数据流

CS: 控制流

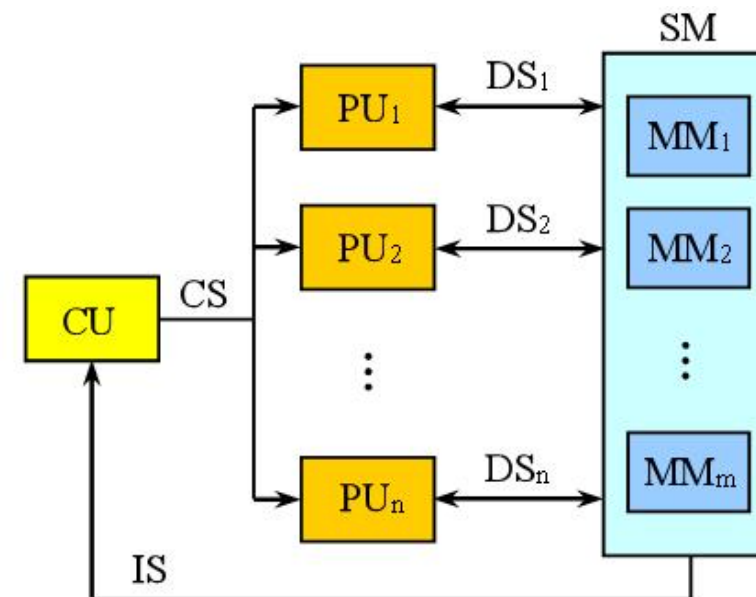
CU: 控制部件

PU: 处理部件

MM和**SM**: 存储器



SISD计算机



SIMD计算机

1.1.4 计算机系统结构的分类

第 42 页

- 类型三：多指令流单数据流 MISD
(Multiple Instruction stream Single Data stream)

$$m \geq 2, n = 1$$

没有实际机器

- 类型四：多指令流多数据流 MIMD
(Multiple Instruction stream Multiple Data stream)

$$m \geq 2, n \geq 2$$

IS: 指令流

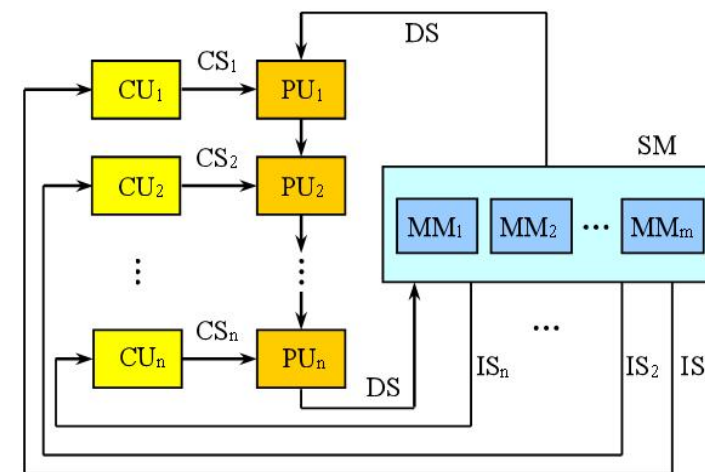
DS: 数据流

CS: 控制流

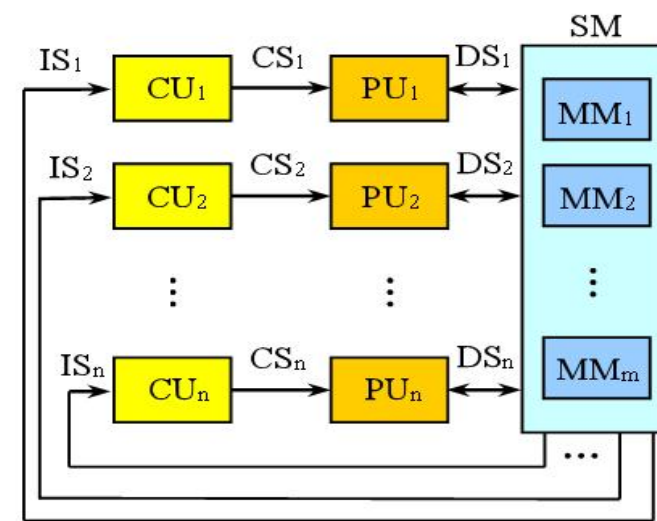
CU: 控制部件

PU: 处理部件

MM和SM: 存储器



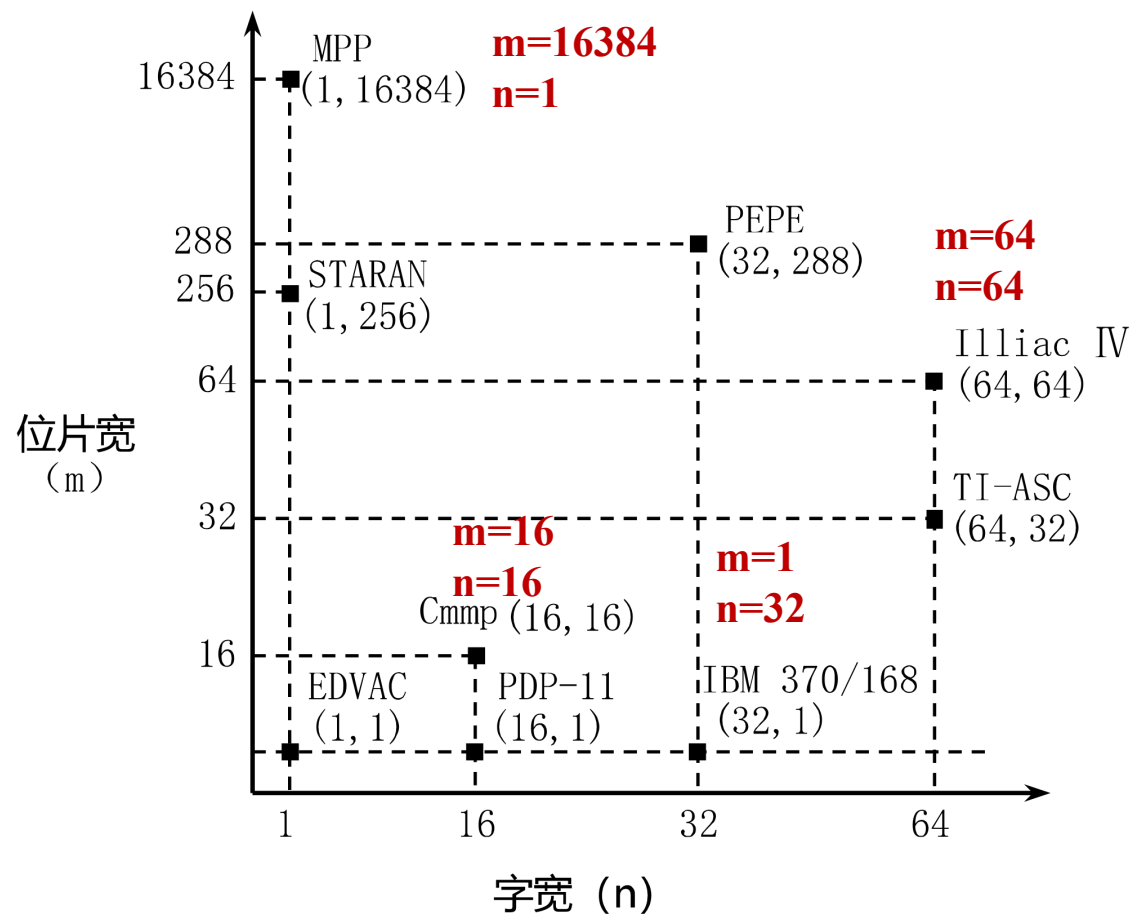
MISD计算机



MIMD计算机

1.1.4 计算机系统结构的分类

第 43 页



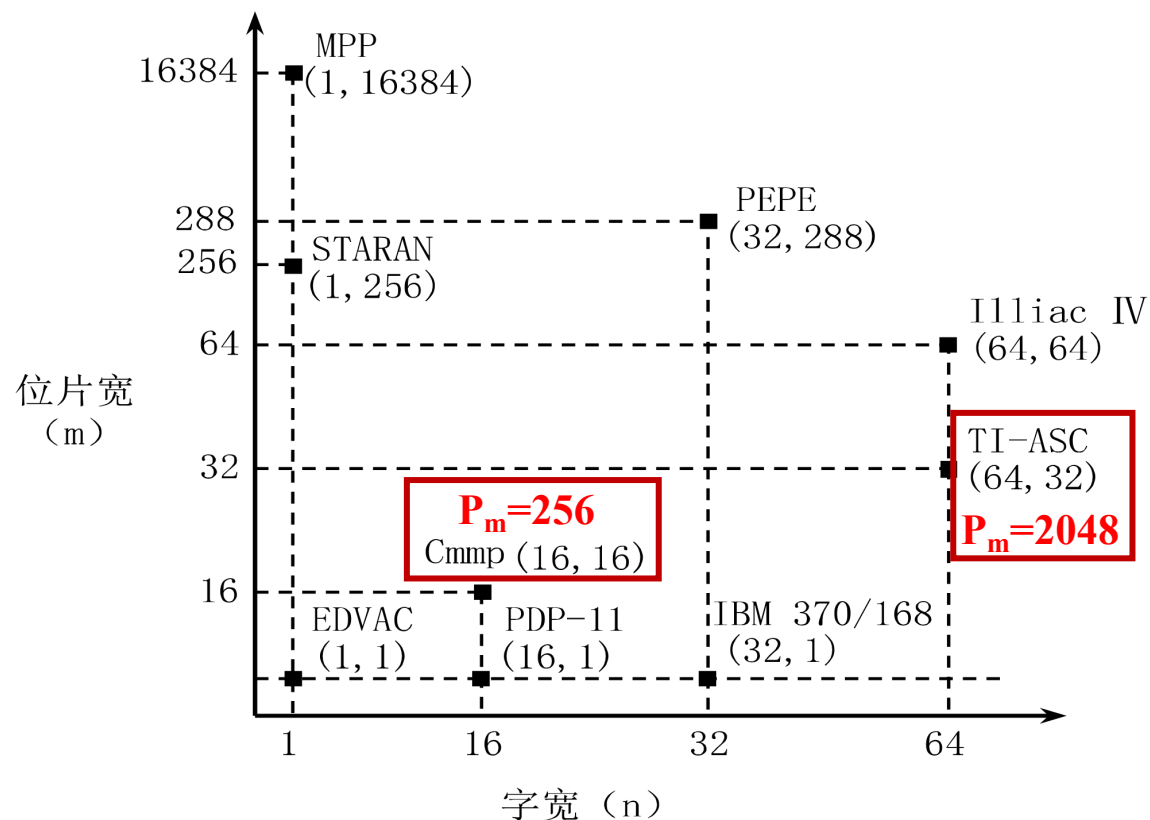
第二种：冯氏分类法

- 最大并行度 P_m ：计算机系统在单位时间内能够处理的最大二进制位数。
 - 用平面直角坐标系中的一个点代表一个计算机系统
 - 横坐标表示字宽 (n位)
 - 纵坐标表示一次能同时处理的字数 (m字)。
 - $m \times n$ 就表示了其最大并行度。

最大并行度 $P_m =$ 同时处理时一个字中的二进制位数 $n \times$ 同时处理的字数 m

1.1.4 计算机系统结构的分类

第 44 页



■ 根据最大并行度将计算机系统结构分成4类：

◆ **字串位串**： $n = 1, m = 1$ 。（第一代计算机发展初期的纯串行计算机）

◆ **字串位并**： $n > 1, m = 1$ 。这是传统的单处理机，同时处理单个字的多个位，如16位、32位等。

◆ **字并位串**： $n = 1, m > 1$ 。同时处理多个字的同一位（位片）。

◆ **字并位并**： $n > 1, m > 1$ 。同时处理多个字的多个位。

1.1.4 计算机系统结构的分类

第 45 页

- **平均并行度**：与最大并行度密切相关的一个指标，取决于系统的运用程度，与应用程序有关。

假设每个时钟周期内能同时处理的二进制位数为 P_i ，则 T 个时钟周期内的平均并行度为：

$$P_a = \frac{\sum_{i=1}^T P_i}{T}$$

系统在 T 个时钟周期内的**平均利用率**定义为：

$$\mu = \frac{P_a}{P_m} = \frac{\sum_{i=1}^T P_i}{TP_m}$$

1.1.4 计算机系统结构的分类

第 46 页

第三种：Handler分类法

- 根据并行度和流水线对计算机进行分类。
- 把计算机的硬件结构分成3个层次
 - 程序控制部件 (PCU) 的个数 k
 - 算术逻辑部件 (ALU) 或处理部件 (PE) 的个数 d
 - 每个算术逻辑部件包含基本逻辑线路(ELC)的套数 w
- 用公式表示：
$$t(\text{系统型号}) = (k, d, w)$$
- 进一步改进：
$$t(\text{系统型号}) = (k \times k', d \times d', w \times w')$$

k' : 宏流水线中程序控制部件的个数; d' : 指令流水线中算术逻辑部件的个数;
 w' : 操作流水线中基本逻辑线路的套数

1.1.4 计算机系统结构的分类

第 47 页

例如：Cray-1有1个CPU，12个相当于ALU或PE的处理部件，可以最多实现8级流水线。字长为64位，可以实现1~14位流水线处理。所以Cray-1系统结构可表示为：

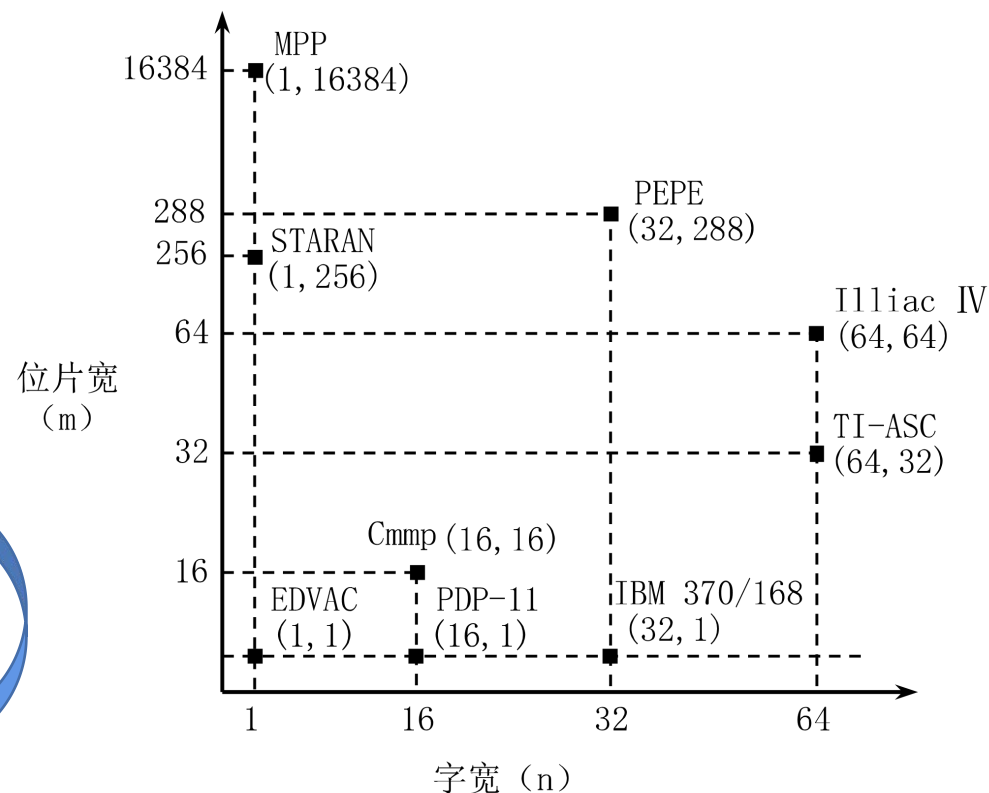
$$t(\text{Cray-1}) = (1, 12 \times 8, 64 \times (1 \sim 14))$$

几个例子：

$$\begin{aligned} t(\text{PDP-11}) &= (1, 1, 16) \\ t(\text{Illiac IV}) &= (1, 64, 64) \\ t(\text{STARAN}) &= (1, 8192, 1) \\ t(\text{PEPE}) &= (1 \times 3, 288, 32) \\ t(\text{TI-ASC}) &= (1, 4, 64 \times 8) \end{aligned}$$

PEPE计算机以一台 CDC7600计算机为主机，每个单元包括三台处理机，周围有288个处理单元，字长为32位。

TI ASC (1972年)是世界上第一批向量巨型计算机之一。



1.2 计算机系统的设计

1.2.1 计算机系统设计的定量原理

第 49 页

1970s



Intel® 4004 Processor
问世 1971



Intel® 8008 Processor
问世 1972



Intel® 8080 Processor
问世 1974



Intel® 8086 Processor
问世 1978



Intel® 8088 Processor
问世 1979

1980s



Intel® 286 Processor
问世 1982



Intel® 386 Processor
问世 1985



Intel® 486 Processor
问世 1989

1990s



Intel® Pentium®
Processor
问世 1993



Intel® Pentium®
Pro Processor
问世 1995



Intel® Pentium® II
Processor
问世 1997



Intel® Celeron®
Processor
问世 1998



Intel® Pentium®
III Processor
问世 1999

2000s



Intel® Pentium® 4
Processor
问世 2000



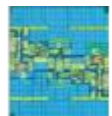
Intel® Itanium®
Processor
问世 2001



Intel® Xeon®
Processor
问世 2001



Intel® Itanium® 2
Processor
问世 2003



Intel® Dual-Core
Itanium® 2 Processor
问世 2006



Intel® Core™2
Duo Processor
问世 2006

性能不断提升

关键问题：如何优化计算机性能与如何判定优化效果？

1.2.1 计算机系统设计的定量原理

第 50 页

一个简单程序，假设完成一个加法指令需2ns，完成一个乘法指令需要10ns

ADD 指令
ADD 指令
.....
ADD 指令
MUL 指令

} 99条加法指令
→ 1条乘法指令

$$\text{总执行时间 } T = 99 \times 2 + 1 \times 10 = 208 \text{ ns}$$

方案一：优化加法指令，使完成一个加法指令仅需1ns

$$\text{总执行时间 } T_1 = 99 \times 1 + 1 \times 10 = 109 \text{ ns}$$

方案二：优化乘法指令，使完成一个乘法指令仅需5ns

$$\text{总执行时间 } T_2 = 99 \times 2 + 1 \times 5 = 203 \text{ ns}$$

**经常性事件
原则**

1.2.1 计算机系统设计的定量原理

第 51 页

1. 以经常性事件为重点

■ 以经常性事件为重点是计算机设计中最重要、最广泛采用的设计原则

➤ 原因：对经常性事件采用优化方法能得到更多的总体上的改进

➤ 优化：分配更多的资源以达到更高的性能

■ 一般来说，经常性事件的处理比较简单，因此也就更容易进行优化实现。

ADD 指令
ADD 指令
.....
ADD 指令
MUL 指令

99条加法指令

1条乘法指令

总执行时间 $T=99 \times 2 + 1 \times 10 = 208\text{ns}$

优化加法指令

总执行时间 $T_1=99 \times 1 + 1 \times 10 = 109\text{ns}$

思考：具体改进的方法？

1.2.1 计算机系统设计的定量原理

第 52 页



吉恩·阿姆达尔
IBM 360计算机设计
师，提出了计算机系
统结构

2. Amdahl定律——评估方案优化效果

加快某部件执行速度所能获得的系统性能加速比，受限于该部件的执行时间占系统中总执行时间的百分比。

系统性能加速比：

$$\text{加速比} = \frac{\text{系统性能}_{\text{改进后}}}{\text{系统性能}_{\text{改进前}}} = \frac{\text{总执行时间}_{\text{改进前}}}{\text{总执行时间}_{\text{改进后}}}$$

1.2.1 计算机系统设计定量原理

第 53 页

■加速比依赖于两个因素：

➤ **可改进比例 (F_e)**：在改进前的系统中，可改进部分的执行时间在总的执行时间中所占的比例。 **它总是小于等于1。**

例如：一个需运行60秒的程序中有20秒的运算可以加速，那么这个比例就是20/60。

➤ **部件加速比 (S_e)**：可改进部分改进以后性能提高的倍数。它是改进前所需的执行时间与改进后执行时间的比。 **一般情况下部件加速比是大于1的。**

例如：若系统改进后，可改进部分的执行时间是2秒，而改进前其执行时间为5秒，则部件加速比为5/2。

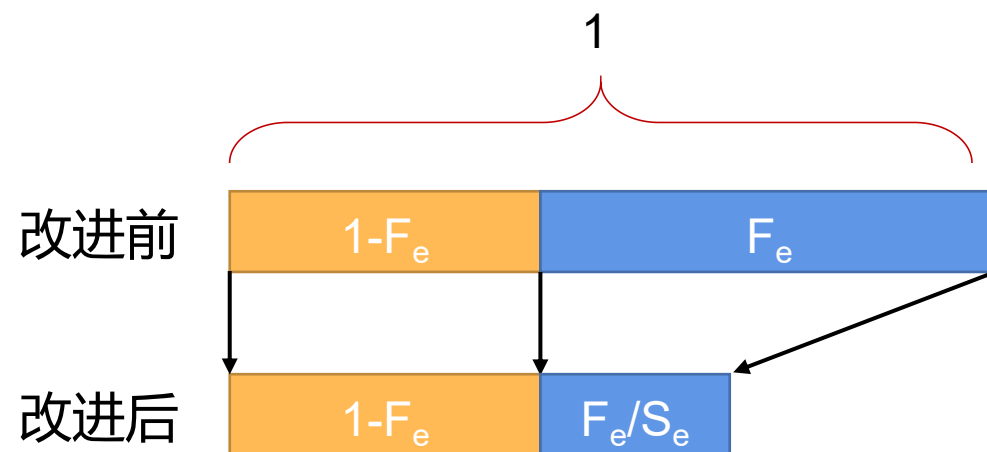
1.2.1 计算机系统设计的定量原理

第 54 页

改进后程序的总执行时间 T_n

$$T_n = T_0 \left(1 - F_e + \frac{F_e}{S_e} \right)$$

- T_0 : 改进前整个程序的执行时间
- $1 - F_e$: 不可改进比例



系统加速比 S_n 为改进前与改进后总执行时间之比:

$$S_n = \frac{T_0}{T_n} = \frac{1}{(1 - F_e) + \frac{F_e}{S_e}}$$

1.2.1 计算机系统设计的定量原理

第 55 页

例1.1 将计算机系统中某一功能的处理速度加快到**15**倍，但该功能的处理时间仅占整个系统运行时间的40%，则采用此增强功能方法后，能使整个系统的性能提高多少？

解：由题可知： $F_e = 40\% = 0.4$, $S_e = 15$, 根据Amdahl定律可知：

$$S_n = \frac{1}{(1 - F_e) + \frac{F_e}{S_e}} = \frac{1}{(1 - 0.4) + \frac{0.4}{15}} = \frac{1}{0.6 + 0.027} \approx 1.6$$

采用此增强功能方法后，能使整个系统的性能提高到原来的1.6倍。

1.2.1 计算机系统设计的定量原理

第 56 页

例1.2 将计算机系统中某一功能的处理速度加快到**30**倍，但该功能的处理时间仅占整个系统运行时间的40%。

问题1：采用此增强功能方法后，能使整个系统的性能提高多少？

解（1）：由题可知： $F_e = 40\% = 0.4$, $S_e = 30$, 根据Amdahl定律可知：

$$S_n = \frac{1}{(1 - F_e) + \frac{F_e}{S_e}} = \frac{1}{(1 - 0.4) + \frac{0.4}{30}} = \frac{1}{0.6 + 0.013} \approx 1.63 \quad \text{略大于1.6}$$

采用此增强功能方法后，能使整个系统的性能提高到原来的1.63倍。

重要推论1：Amdahl定律是一种性能改进的递减规则。但如果仅仅对计算任务中的一部分做性能改进，则改进得越多，所得到的总体性能的提升就越有限。

1.2.1 计算机系统设计的定量原理

第 57 页

例1.2 将计算机系统中某一功能的处理速度加快到**30**倍，但该功能的处理时间仅占整个系统运行时间的**40%**。

问题1：采用此增强功能方法后，能使整个系统的性能提高多少？

问题2：如果只针对某一功能优化，能使整个系统的性能加速比最大值多少？

解（2）：由题可知： $F_e = 40\% = 0.4$ ，根据Amdahl定律可知：

$$S_n = \frac{1}{(1 - F_e) + \frac{F_e}{S_e}} \quad \xrightarrow{\text{blue arrow}} \quad S_n(\max) = \frac{1}{1 - F_e} = \frac{1}{1 - 0.4} \approx 1.67$$

$\xrightarrow{\text{red arrow}} \infty$

重要推论2：如果只针对整个任务的一部分进行改进和优化，那么所获得的加速比不超过 $1 / (1 - F_e)$

1.2.1 计算机系统设计的定量原理

第 58 页

例1.3 某计算机系统采用浮点运算部件后，使浮点运算速度提高到原来的**25倍**，而系统运行某一程序的整体性能提高到原来的**4倍**，试计算该程序中浮点操作所占的比例。

解：由题可知： $S_e = 25$, $S_n = 4$, 根据Amdahl定律可知：

$$4 = \frac{1}{(1 - F_e) + \frac{F_e}{25}} \quad \leftarrow \text{判定改进/优化比例}$$

由此可得： $F_e = 78.1\%$, 即程序中浮点操作所占的比例为78.1%。

1.2.1 计算机系统设计的定量原理

第 59 页

例1.4 某计算机系统运行某一程序，其中浮点操作所占的比例为**80%**。如果采用浮点运算部件优化浮点操作后，系统运行某一程序的整体性能提高到原来的**4倍**，试计算至少使浮点运算速度提高到原来的多少倍？

解：由题可知： $F_e = 80\%$, $S_n = 4$, 根据Amdahl定律可知：

$$4 = \frac{1}{(1 - 0.8) + \frac{0.8}{S_e}}$$

判定改进/优化目标

思考：2及以上
可优化操作，如
何计算？

由此可得： $S_e = 16$, 即至少使浮点运算速度提高到原来的16倍。

1.2.1 计算机系统设计的定量原理

第 60 页

设计优化的原则：以经常性事件为重点 (F_e 、 S_e 值尽可能大)

- F_e ：对经常发生的情况采用优化方法的原则进行选择，以得到更多的总体上的改进。
- S_e ：优化是指分配更多的资源、达到更高的性能或者分配更多的电能等。

$$\text{加速比} = \frac{\text{系统性能}_{\text{改进后}}}{\text{系统性能}_{\text{改进前}}} = \frac{\text{总执行时间}_{\text{改进前}}}{\text{总执行时间}_{\text{改进后}}}$$

← CPU的实际时间

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

mul R0, R1, R2 ; // 乘法指令
add R2, R3, R4 ; // 加法指令

优化

mac R0, R1, R2, R3 ;
// 乘加指令

关键点：操作占比 F_e 变化

在设计阶段如何评估CPU执行时间和优化效果？

3. CPU性能公式

- 执行一个程序所需的CPU时间

$$\begin{aligned}\text{CPU时间} &= \text{执行程序所需的时钟周期数} \times \text{时钟周期时间} \\ &= \text{执行程序所需的时钟周期数} / \text{时钟频率}\end{aligned}$$

其中，时钟周期时间T是系统时钟频率的倒数 $1/f$ 。

- 每条指令执行的平均时钟周期数CPI (Cycles Per Instruction)

$$\text{CPI} = \text{执行程序所需的时钟周期数} / \text{IC}$$

其中，IC表示所执行的指令条数

- 程序执行的CPU时间可以写成

$$\text{CPU时间} = \text{IC} \times \text{CPI} \times \text{时钟周期时间}$$

1.2.1 计算机系统设计的定量原理

第 62 页

■ CPU的性能取决于三个参数:

- 时钟周期时间: 取决于硬件实现技术和计算机组成
- CPI: 取决于计算机组成和指令系统的结构
- IC: 取决于指令系统的结构和编译技术

$$\text{CPU时间} = \text{IC} \times \text{CPI} \times \text{时钟周期时间}$$

$$= \text{执行程序所需的时钟周期数} \times \text{时钟周期时间}$$

$$= \sum_{i=1}^n (\text{CPI}_i \times \text{IC}_i) \times \text{时钟周期时间}$$

CPU时钟周期数



假设: 计算机系统有n种指令; CPI_i : 第i种指令的处理时间; IC_i : 在程序中第i种指令出现的次数

1.2.1 计算机系统设计的定量原理

第 63 页

$$\text{CPU时钟周期数} = \sum_{i=1}^n (\text{CPI}_i \times \text{IC}_i)$$

CPI可以表示为：

$$\text{CPI} = \frac{\text{CPU时钟周期数}}{\text{IC}} = \frac{\sum_{i=1}^n (\text{CPI}_i \times \text{IC}_i)}{\text{IC}} = \sum_{i=1}^n (\text{CPI}_i \times \frac{\text{IC}_i}{\text{IC}})$$

其中： $\frac{\text{IC}_i}{\text{IC}}$ 反映了第*i*种指令在程序中所占的比例。



保持不变？

1.2.1 计算机系统设计的定量原理

第 64 页

例1.5 假设FP指令的比例为**25%**，其中，FPSQR占全部指令的比例为**2%**，FP操作的CPI为**4**（FPSQR操作的CPI为**20**），其他指令（除FP指令）的平均CPI为**1.33**。现有两种改进方案：第一种是把FPSQR操作的CPI减至**2**，第二种是把所有的FP操作的CPI减至**2**，试比较两种方案对系统性能的提高程度。

解：没有改进之前，每条指令的平均时钟周期CPI为：

$$CPI = \sum_{i=1}^n \left(CPI_i \times \frac{IC_i}{IC} \right) = (4 \times 25\%) + (1.33 \times 75\%) = 2$$

1.2.1 计算机系统设计的定量原理

第 65 页

(1) 采用第一种方案

FPSQR操作的CPI由 $CPI_{FPSQR} = 20$ 减至 $CPI'_{FPSQR} = 2$ ，则整个系统的指令平均时钟周期数为：

$$\begin{aligned} CPI_1 &= CPI_{FP} \times 75\% + CPI_{FPSQR} \times 23\% + CPI'_{FPSQR} \times 2\% \\ &= CPI_{FP} \times 75\% + CPI_{FPSQR} \times 23\% + \underbrace{CPI_{FPSQR} \times 2\% - CPI_{FPSQR} \times 2\%}_{\text{blue line}} + \underbrace{CPI'_{FPSQR} \times 2\%}_{\text{red line}} \\ &= CPI - (CPI_{FPSQR} - CPI'_{FPSQR}) \times 2\% \\ &= 2 - (20 - 2) \times 2\% = 1.64 \end{aligned}$$

(2) 采用第二种方案

所有FP操作的CPI由 $CPI_{FP} = 4$ 减至 $CPI'_{FP} = 2$ ，则整个系统的指令平均时钟周期数为：

$$\begin{aligned}CPI_2 &= CPI - (CPI_{FP} - CPI'_{FP}) \times 25\% \\ &= 2 - (4 - 2) \times 25\% = 1.5\end{aligned}$$

从降低整个系统的指令平均时钟周期数的程度来看，**第二种方案优于第一种方案。**

结论：一般情况下，优化的比例越大，优化的效果越好！

1.2.1 计算机系统设计的定量原理

第 67 页

例1.6 考虑条件分支指令的两种不同设计方法：

- (1) CPU₁：通过比较指令设置条件码，然后测试条件码进行分支。
- (2) CPU₂：在分支指令中包括比较过程。

在这两种CPU中，条件分支指令都占用2个时钟周期，而所有其它指令占用1个时钟周期。对于CPU₁，执行的指令中分支指令占30%；由于每条分支指令之前都需要有比较指令，因此比较指令也占30%。由于CPU₁在分支时不需要比较，因此CPU₂的时钟周期时间是CPU₁的1.35倍。问：哪一个CPU更快？如果CPU₂的时钟周期时间只是CPU₁的1.15倍，哪一个CPU更快呢？

1.2.1 计算机系统设计的定量原理

第 68 页

解:我们不考虑所有系统问题, 所以可用CPU性能公式。占用2个时钟周期的分支指令占总指令的30%, 剩下的指令占用1个时钟周期。所以

$$CPI_1 = 0.3 \times 2 + 0.70 \times 1 = 1.3$$

则CPU₁性能为:

$$\text{总CPU时间}_1 = IC_1 \times 1.3 \times \text{时钟周期}_1$$

根据假设, 有:

$$\text{时钟周期}_2 = 1.35 \times \text{时钟周期}_1$$

在CPU₂中没有独立的比较指令, 所以CPU₂的程序量为CPU₁的70%, 分支指令的比例为:

$$30\%/70\% = 42.8\%$$

1.2.1 计算机系统设计的定量原理

第 69 页

这些分支指令占用2个时钟周期，而剩下的57.2%的指令占用1个时钟周期，因此：

$$CPI_2 = 0.428 \times 2 + 0.572 \times 1 = 1.428$$

因为CPU₂不执行比较，故：

$$IC_2 = 0.7 \times IC_1$$

因此CPU₂性能为：

$$\begin{aligned} \text{总CPU时间}_2 &= IC_2 \times CPI_2 \times \text{时钟周期}_2 \\ &= 0.7 \times IC_1 \times 1.428 \times (1.35 \times \text{时钟周期}_1) \\ &= 1.349 \times IC_1 \times \text{时钟周期}_1 \end{aligned}$$

架构优化相对优化时，集成电路工艺优化的影响也很大



在这些假设之下，尽管CPU₂执行指令条数较少，CPU₁因为有着更短的时钟周期，所以比CPU₂快。

1.2.1 计算机系统设计的定量原理

第 70 页

如果CPU₂的时钟周期时间仅仅是CPU₁的1.15倍，则

$$\text{时钟周期}_2 = 1.15 \times \text{时钟周期}_1$$

CPU₂的性能为：

$$\begin{aligned}\text{总CPU时间}_2 &= IC_2 \times CPI_2 \times \text{时钟周期}_2 \\ &= 0.7 \times IC_1 \times 1.428 \times (1.15 \times \text{时钟周期}_1) \\ &= 1.15 \times IC_1 \times \text{时钟周期}_1\end{aligned}$$

因此CPU₂由于执行更少指令条数，比CPU₁运行更快。



**集成电路工艺
优化和计算机
系统结构优化
的作用更大**

1.2.2 计算机系统设计者的主要任务

第 71 页

- 计算机系统设计者的目标:设计出能满足用户的功能需求、有较长的生命周期、且又具有很高的性能价格比的系统;

- 计算机系统设计者的任务包括:

指令系统的设计 → 数据表示的设计 → 功能的组织 → 逻辑设计 → 物理实现

- 设计一个计算机系统大致要完成三个方面的工作:

1.确定用户对计算机系统的功能、价格和性能的要求

- 应用领域: 专用还是通用? 面向科学计算还是面向商用处理?
- 软件兼容: 指一台计算机上的程序不加修改就可以搬到另一台计算机上正常运行。

- 操作系统需求: 包括地址空间大小、存储管理、保护等操作系统的需求提供支持, 是很重要的一点。
- 参考标准: 确定系统中哪些方面要采用标准以及采用什么标准, 如浮点数标准、I/O总线标准、网络标准、程序设计语言标准等。

2. 软硬件功能分配

- 考虑如何优化设计？必须考虑软硬件功能的合理分配。
- 软件和硬件在实现功能上是等价的
 - ✓ 用软件实现的优点：设计容易、修改简单，且可减少硬件成本，但所实现的功能的速度较慢；
 - ✓ 用硬件实现的优点：速度快、性能高，但它修改困难，灵活性差。
- 在软硬件之间进行折中和取舍。

`_mm_pcmpestri(_m128 a, _m128 b)`

一次对一组16个字符与另一组字符作比较，也就是说一个指令可以作最多 $16 \times 16 = 256$ 次比较

3. 设计出生命周期长的系统结构

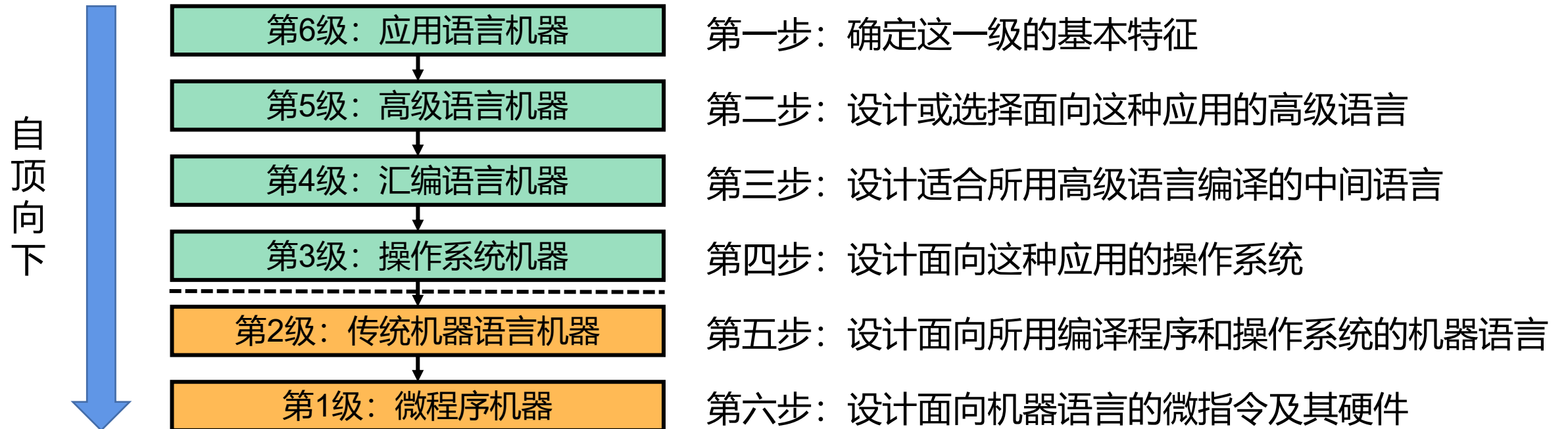
- 特别注意计算机应用和计算机技术的发展趋势
- 设计出具有一定前瞻性的系统结构，以使得它具有较长的生命周期。

1.2.3 计算机系统设计的主要方法

第 73 页

1. “自顶向下”或“由上往下”(top-down)设计

- 从层次结构中的最上面一级开始，逐层往下设计各层的机器。
- 首先，确定面对使用者的那级机器的基本特征、数据类型和格式、基本命令等。然后，再逐级往下设计，每级都考虑如何优化上一级的实现。



1.2.3 计算机系统设计的主要方法

第 74 页

2. “自底向上”或“由下往上” (bottom-up) 设计

- 从层次结构的最下面一级开始，逐层往上设计各层的机器。
- 采用这种方法时，软件技术完全处于被动状态，这会造成软件和硬件的脱节，使整个系统的效率降低。

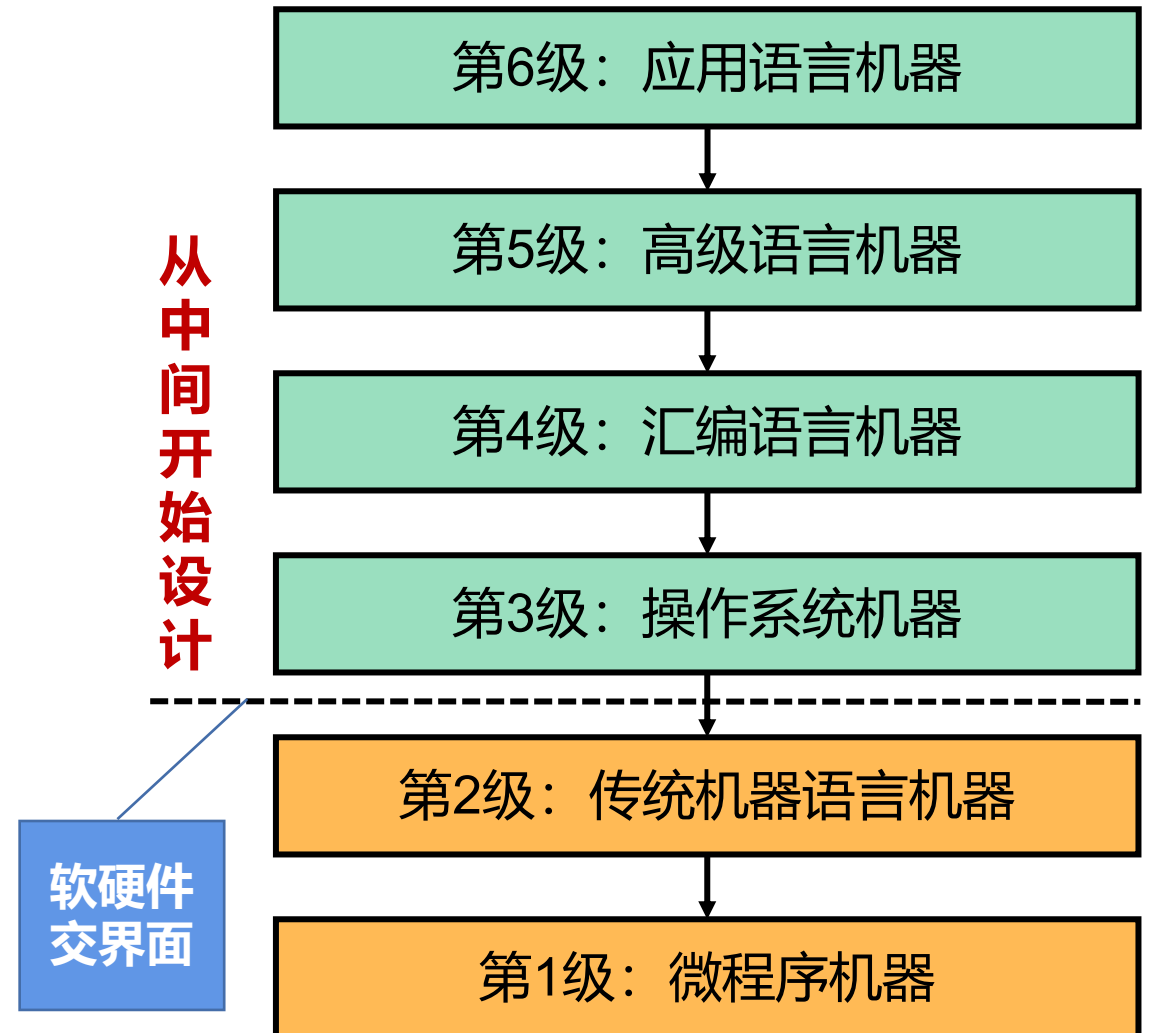
(在早期被采用得比较多，现在已经很少被采用了)

1.2.3 计算机系统设计的主要方法

第 75 页

3. “从中间开始” (middle-out) 设计

- “由上往下”和“由下往上”设计方法的主要缺点：**软件与硬件设计分离和脱节**
- **解决方法：**综合考虑软、硬件的分工，从中间开始设计。“**中间**”：层次结构中的软硬件的交界面，目前一般是在传统机器语言机器级与操作系统机器级之间。
- **从中间开始设计：**首先要进行软、硬件功能分配，确定好这个界面；然后从这个界面开始，软件设计者开始往上设计操作系统、汇编、编译系统等；硬件设计者开始往下设计传统机器级、微程序机器级等。



1.3 计算机系统的性能评测

1.3 计算机系统的性能评测

第 77 页

假设两台计算机X和Y，判定哪一台机器性能更好的关键是确定性能指标评价方法。

$$\text{性能比}n = \frac{\text{性能X}}{\text{性能Y}} \quad \left\{ \begin{array}{l} n>1 \text{表示计算机X的性能高于计算机Y的性能} \\ n=1 \text{表示计算机X的性能与计算机Y的性能相当} \\ n<1 \text{表示计算机X的性能低于计算机Y的性能} \end{array} \right.$$

计算机技术指标：

- 峰值性能：≥1600GMACS, 300 GFLOPS
- 通信速率≥125Gbps

直接数值量化：
物理数值，由处理器架构决定

1.3 计算机系统的性能评测

第 78 页

MIPS（每秒百万条指令数）定义为：

$$\text{MIPS} = \frac{\text{指令条数}}{\text{执行时间} \times 10^6} = \frac{\text{时钟频率}}{\text{CPI} \times 10^6}$$

MFLOPS（每秒百万次浮点操作次数）定义为：

$$\text{MFLOPS} = \frac{\text{程序中的浮点操作次数}}{\text{执行时间} \times 10^6}$$

TOPS（每秒万亿操作次数）定义为：

$$\text{TOPS} = \frac{\text{处理器的操作次数}}{\text{时间} \times 10^{12}}$$

计算公式： $\text{IPC} \times f_{\text{HZ}} \times n_c$

IPC：每周期执行指令数 $1/\text{CPI}$

f_{HZ} ：机器工作频率

n_c ：处理器核数（并行度）

某机器性能：

- 最高可达64TOPS
- 典型功耗20W

哪一个实际应用效果更好？

某机器性能：

- 最高可达48TOPS
- 典型功耗15W

1.3 计算机系统的性能评测

第 79 页

衡量计算机性能的唯一且可靠的标准是真实程序的执行时间

1. 执行时间和吞吐率：如何评测一台计算机的性能，与测试者看问题的角度有关。

- 用户关心的是：**单个程序的执行时间**（执行单个程序所花的时间很少）
- 数据处理中心的管理人员关心的是：**吞吐率**（在单位时间里能够完成的任务很多）

假设两台计算机为X和Y，X比Y快的意思是：对于给定任务，X的执行时间比Y的执行时间少。

X的性能是Y的n倍：
$$\frac{\text{执行时间Y}}{\text{执行时间X}} = n$$

执行时间与性能成反比：
$$n = \frac{\text{执行时间Y}}{\text{执行时间X}} = \frac{\frac{1}{\text{性能Y}}}{\frac{1}{\text{性能X}}} = \frac{\text{性能X}}{\text{性能Y}}$$

如何计算执行时间？

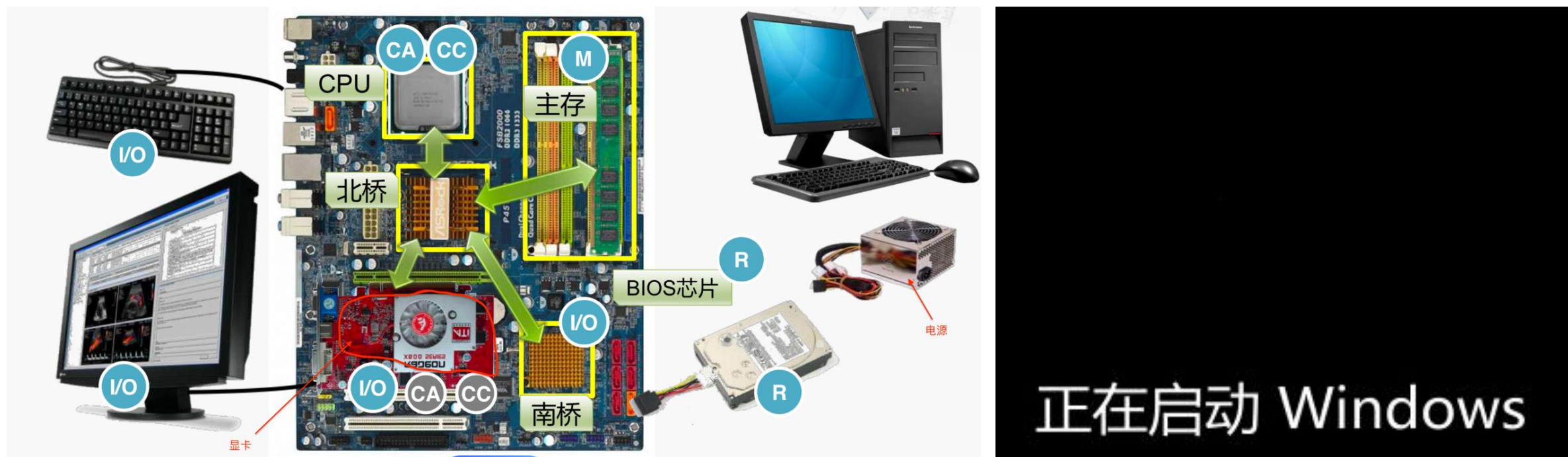
1.3 计算机系统的性能评测

第 80 页

执行时间的计算:

1. 计算机完成某一任务所花费的**全部时间T**

执行时间 T =CPU时间+磁盘访问时间+存储器访问+输入/输出开销+操作系统开销



1.3 计算机系统的性能评测

第 81 页

2. CPU时间：CPU执行所给定的程序所花费的时间，不包含I/O等待时间以及运行其它程序的时间。

- **用户CPU时间**：用户程序所耗费的CPU时间。
- **系统CPU时间**：用户程序运行期间操作系统所耗费的CPU时间。



如何公平公正
确定用户程序？

2. 基准测试程序

- 解决方法：把应用程序中用得最频繁的那部分核心程序或人工合成出来的程序作为评价计算机性能的标准程序，称为基准测试程序（Benchmarks）。
 - 桌面机/服务器的基准程序：SPEC (www.spec.org) (System Performance Evaluation Cooperative) 是由30个左右世界知名计算机大厂商所支持的非盈利的合作组织。
 - Whetstone与Dhrystone是最流行的合成测试程序。例如，Dhrystone是由Reinhold P. Weicker在1984年提出来的一个基准测试程序，其主要目的是测试处理器的整数运算和逻辑运算的性能。Dhrystone标准的测试方法很简单，就是单位时间内跑了多少次Dhrystone程序，其指标单位为DMIPS/MHz。例如，ARM公司的Cortex-M7处理器性能为2.14DMIPS/MHz。

1.3 计算机系统的性能评测

第 83 页

- 问题：性能测试结果除了和采用什么测试程序有关以外，还和在什么条件下进行测试有关。
- 基准测试程序设计者对制造商的要求：
 - 采用同一种语言编译器；
 - 对同一种语言的程序都采用相同的一组编译标志。
- 一个问题：是否允许修改测试程序的源程序，则有三种不同的处理方法：
 - 不允许修改；
 - 允许修改，但因测试程序很复杂或者很大，几乎是无法修改；
 - 允许修改，只要保证最后输出的结果相同。
- 基准测试程序套件：由各种不同的真实应用程序构成，能比较全面地反映计算机在各个方面的处理性能。

1.3 计算机系统的性能评测

第 84 页

- 桌面计算机的基准测试程序套件分为两大类：处理器性能测试程序，图形性能测试程序
- SPEC89：用于测试处理器性能。10个程序（4个整数程序，6个浮点程序）
- 演化出了多个版本：SPEC89→SPEC92→SPEC95→SPEC2000→SPEC2006→SPEC2017

Benchmark	Type	Source	Description
gzip	Integer	C	Compression using the Lempel-Ziv algorithm
vpr	Integer	C	FPGA circuit placement and routing
gcc	Integer	C	Consists of the GNU C compiler and machine code.
mcf	Integer	C	Combinatorial
crafty	Integer	C	Chess play
parser	Integer	C	Syntactic
eon	Integer	C++	Graphics
perlmbk	Integer	C	Perl benchmarks
gap	Integer	C	Integer
vortex	Integer	C	An object-oriented
bzip2	Integer	C	A block sorting compressio
twolf	Integer	C	Timberwolf: a simulated annealing algorithm for VLSI place and route
wupwise	FP	F77	Lattice gauge theory model of quantum chromodynamics.
swim	FP	F77	Solves shallow water equations using finite difference equations.
mgrid	FP	F77	Multigrid solver over 3-dimensional field.
apply	FP	F77	Parabolic and elliptic partial differential equation solver
mesa	FP	C	Three dimensional graphics library
galgel	FP	F90	Computational fluid
art	FP	C	Int
equake	FP	C	Simulation
facerec	FP	C	Face recog
ammp	FP	C	molecular
lucas	FP	F90	Performs pr
fma3d	FP	F90	Finite element
sixtrack	FP	F77	High energy physics
apsi	FP	F77	A meteorological simulation of p

12个整数程序
用于CINT2000
测试

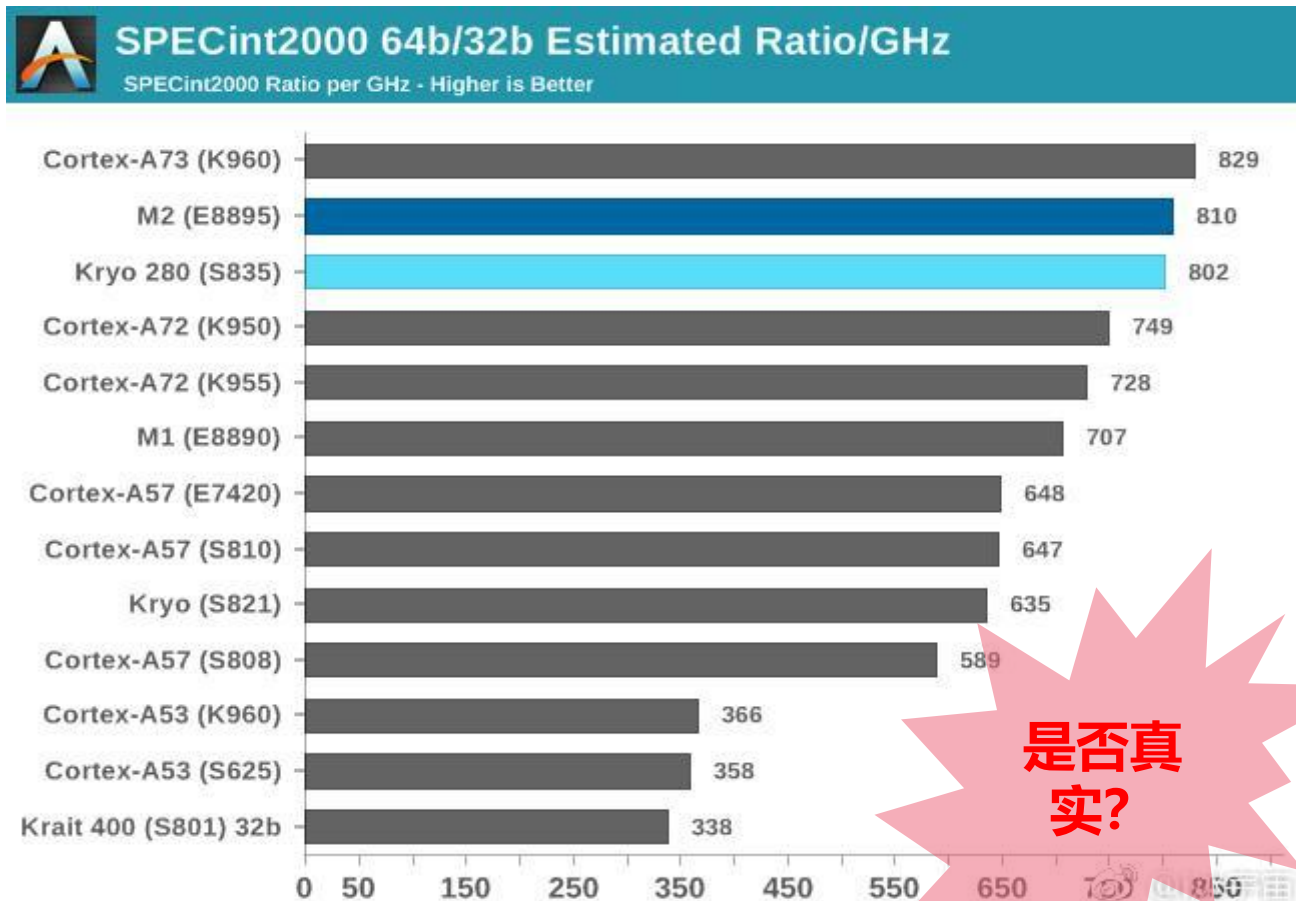
14个浮点程序
用于CFP2000
测试

SPEC测试程序套件中其它系列测试程序组件：

- SPECSFS：NFS（网络文件系统）文件服务器的测试程序。它不仅测试处理器的性能，还可测试I/O系统的性能，重点测试吞吐率。
- SPECWeb：Web服务器测试程序。
- SPECviewperf：用于测试图形系统支持OpenGL库的性能。
- SPECapc：用于测试图形密集型应用的性能。

1.3 计算机系统的性能评测

第 85 页



- 适度性: 评价尺度必须反映处理器执行信号处理算法频度的性能;
- 公正性和精确性: 评价尺度必须来源于基准程序的实际计算结果, 且对任何处理器都是一样的;
- 简单性: 评价尺度必须方便地进行比较, 用一个数值来表示评测结果;
- 广泛性: 评价尺度必须广泛, 适合任何一类可编程处理器;
- 独立性: 评测分值必须经过独立地第三方验证以确保能够真实反映处理器地性能;
- 有效性: 评测分值是有效的, 且是公开的。

1.3 计算机系统的性能评测

第 86 页

	机器A	机器B	机器C
程序P1	1.00	10.00	20.00
程序P2	1000.00	100.00	20.00

从该表可以得出：

■ 执行程序1:

- A机的速度是B机的10倍
- A机的速度是C机的20倍
- B机的速度是C机的2倍

■ 执行程序2:

- B机的速度是A机的10倍
- C机的速度是A机的50倍
- C机的速度是B机的5倍

- 主要问题：计算机的配置基本相同，但不同计算机针对不同测试程序的测试结果是否一致？
- 为了公平地综合比较处理器性能，建立综合度量标准：

◆ 各程序的比重相同时

- 总执行时间
- 算术平均时间
- 调和平均时间

◆ 各程序的比重不同时

- 加权执行时间
- 标准化执行时间

1.3 计算机系统的性能评测

总执行时间

$$S_m = \sum_{i=1}^n T_i$$

算术平均执行时间

$$A_m = \frac{1}{n} \sum_{i=1}^n T_i$$

调和平均执行时间

$$H_m = \frac{n}{\sum_{i=1}^n \frac{1}{T_i}}$$

其中：T_i为第i个程序的执行时间，n为程序数，R_i=1/T_i。

	机器A	机器B	机器C
程序P1	1秒	10秒	20秒
程序P2	1000秒	100秒	20秒
总执行时间S	1001秒	110秒	40秒
算术平均执行时间A	500.5秒	极大值 影响55秒	20秒
调和平均执行时间H	2秒	极小值 影响18秒	20秒

1.3 计算机系统的性能评测

加权算术平均执行时间

$$A_m = \sum_{i=1}^n W_i \times T_i$$

其中：T_i为第i个程序的执行时间，n为程序数，W_i为第i个程序的权重， $\sum_{i=1}^n W_i = 1$

	机器A	机器B	机器C
程序P1	1秒	10秒	20秒
程序P2	1000秒	100秒	20秒
算术平均A ₁	500.5秒	55秒	20秒
算术平均A ₂	91.91秒	18.19秒	20秒
算术平均A ₃	2秒	10.09秒	20秒

	W ₁	W ₂
组合1	0.5	0.5
组合2	0.909	0.091
组合3	0.999	0.001

1.3 计算机系统的性能评测

第 89 页

加权调和平均执行时间

$$H_m = \frac{\sum_{i=1}^n W_i}{\sum_{i=1}^n \frac{W_i}{T_i}}$$

其中： T_i 为第*i*个程序的执行时间， n 为程序数， W_i 为第*i*个程序的权重

	机器A	机器B	处理器C
程序P1	1秒	10秒	20秒
程序P2	1000秒	100秒	20秒
调和平均A ₁	2秒	18.18秒	20秒
调和平均A ₂	1.1秒	10.89秒	20秒
调和平均A ₃	1秒	10.01秒	20秒

	W ₁	W ₂
组合1	0.5	0.5
组合2	0.909	0.091
组合3	0.999	0.001

1.3 计算机系统的性能评测

第 90 页

标准化执行时间的计算方法：先将各程序的执行时间对一台参考机器进行标准化，然后取标准化执行时间的平均值（可以是算术平均值，也可以是几何平均值）

$$A_m = \frac{1}{n} \sum_{i=1}^n ETR_i$$

算术平均

$$G_m = \sqrt[n]{\prod_{i=1}^n ETR_i}$$

几何平均

ETR_i ：为第*i*个程序对参考机器标准后的执行时间， n ：为程序数

1.3 计算机系统的性能评测

	机器A	机器B	机器C
程序P1	1秒	10秒	20秒
程序P2	1000秒	100秒	20秒
总执行时间S	1001秒	110秒	40秒

■ 算术平均值因参考机器不同而不同，几何平均值不因参考机器不同而不同，所以标准化执行时间应采用几何平均值。

	归一化到处理器A			归一化到处理器B			归一化到处理器C		
	A	B	C	A	B	C	A	B	C
程序P1	1.0	10.0	20.0	0.1	1.0	2.0	0.05	0.5	1.0
程序P2	1.0	0.1	0.02	10.0	1.0	0.2	50.0	5.0	1.0
算术平均	1.0	5.05	10.01	5.5	1.0	1.1	25.025	2.75	1.0
几何平均	1.0	1.0	0.63	1.0	1.0	0.63	1.58	1.58	1.0

1.3 计算机系统的性能评测

第 92 页

加权几何平均值 $G_m = \prod_{i=1}^n (ETR_i)^{W_i} = (ETR_1)^{W_1} \times (ETR_2)^{W_2} \times \cdots \times (ETR_n)^{W_n}$

ETR_i：为第i个程序对参考机器标准后的执行时间，**n**：为程序数

G_m表示法有一个很好的特性，**几何平均值的比等于比的几何平均值**

$$\frac{G_m(x_i)}{G_m(y_i)} = G_m\left(\frac{x_i}{y_i}\right)$$

建议课后讨论！

1.4 计算机系统结构的发展

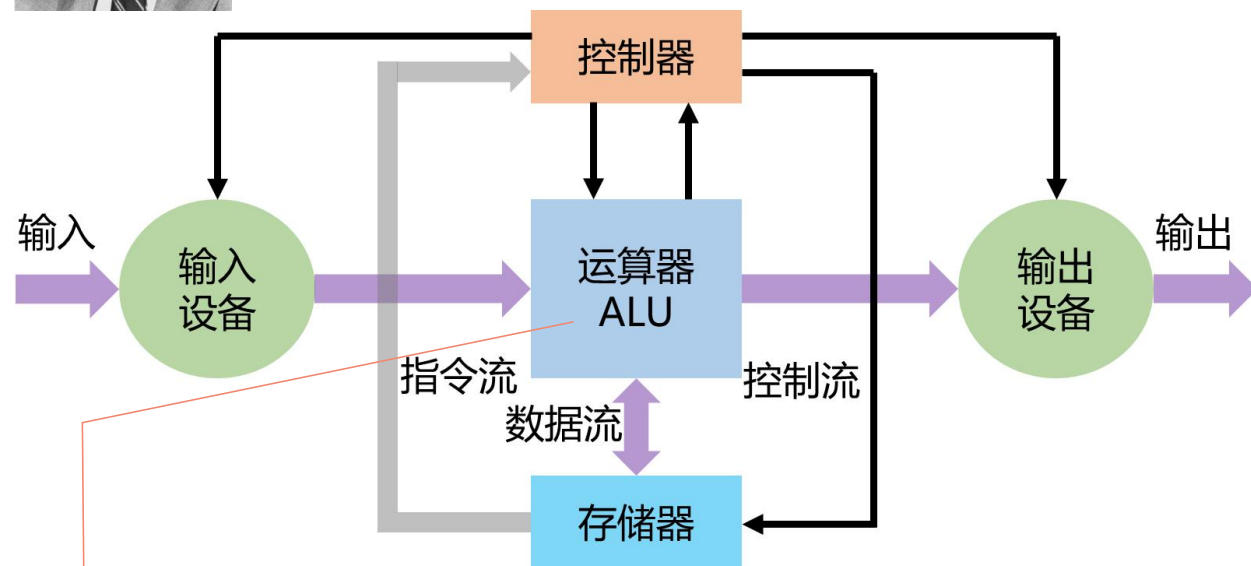
1.4.1 冯·诺依曼结构及其改进

第 94 页



冯·诺依曼
John von Neuman

著名匈牙利裔美籍数学家、计算机科学家、
物理学家和化学家



存储程序计算机的结构

加法、乘法、除法、平方根、求
余数、向量加、向量乘等

存储程序工作原理

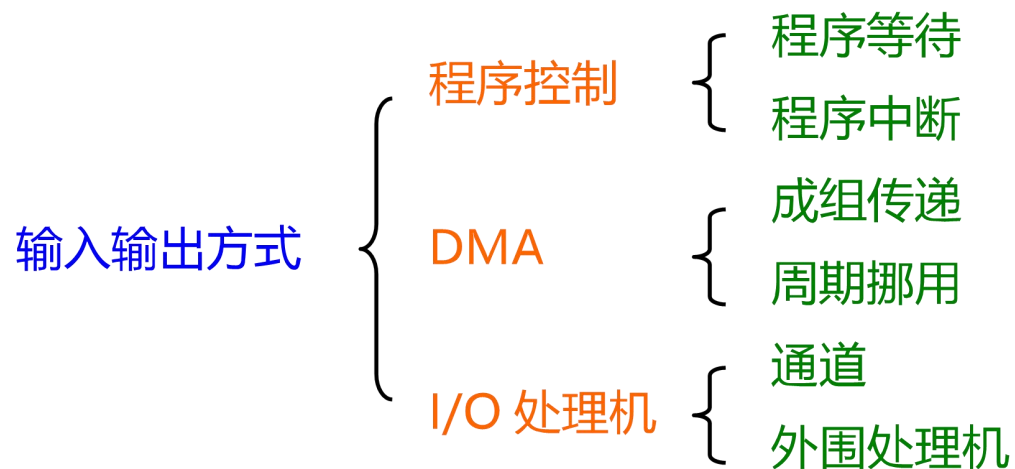
- 指令驱动：将计算过程描述为由许多条指令按一定顺序组成的程序，并放入存储器保存
- 计算机以运算器为中心；在存储器中，指令和数据同等对待，即指令和数据一样可以进行运算，即由指令组成的程序是可以修改的。
- 存储器是按地址访问、按顺序线性编址的一维结构，每个单元的位数是固定的。
- 指令的执行是顺序的，即一般是按照指令在存储器中存放的顺序执行；程序的分支由转移指令实现；由指令计数器PC指明当前正在执行的指令在存储器中的地址。
- 指令由操作码和地址码组成，指令和数据均以二进制编码表示，采用二进制运算。

1.4.1 冯·诺依曼结构及其改进

第 95 页

■ 对系统结构进行的改进

□ 输入/输出方式的改进



□ 采用并行处理技术

- 如何挖掘传统机器中的并行性？
- 在不同的级别采用并行技术。

例如：微操作级、指令级、线程级、进程级、任务级等。

□ 存储器组织结构的发展

- 相联存储器与相联处理机
- 通用寄存器组
- 高速缓冲存储器Cache

□ 指令系统的发展，两个发展方向：

- 复杂指令集计算机CISC
- 精简指令集计算机RISC

1.4.2 软件对系统结构的影响

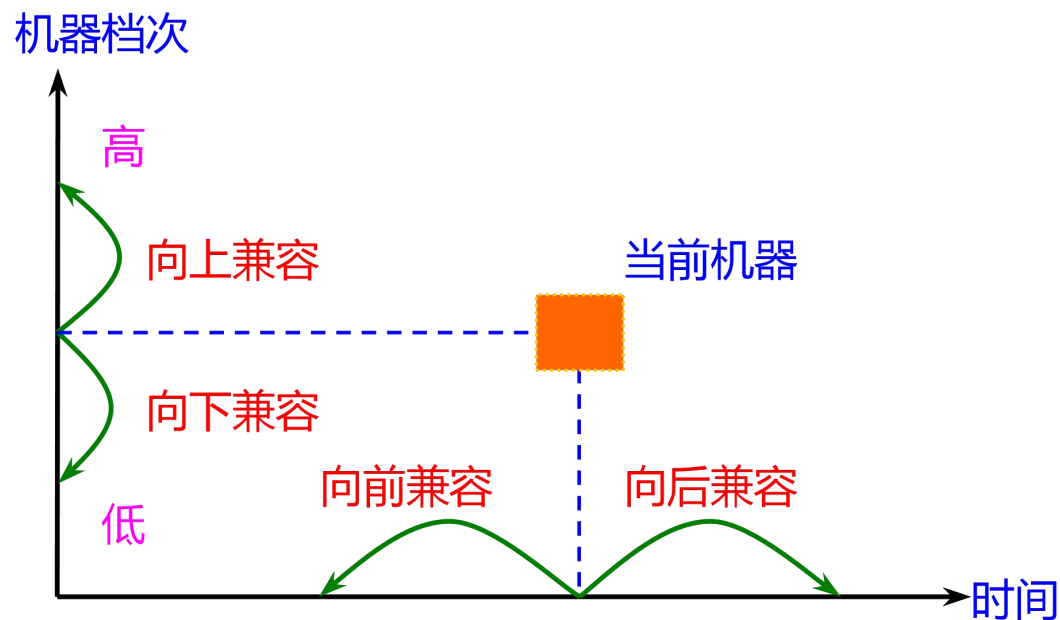
第 96 页

- 软件的可移植性：一个软件可以不经修改或者只需少量修改就可以由一台机器移植到另一台机器上正确地运行。差别只是执行时间的不同，我们称这两台机器是软件兼容的。
- 实现可移植性的常用方法：采用系列机、模拟与仿真、统一高级语言。
 1. **统一高级语言**：实现软件移植的一种理想的方法，但较难实现
 2. **系列机**：由同一厂家生产的具有相同的系统结构，但具有不同组成和实现的一系列不同型号的机器。

1.4.2 软件对系统结构的影响

第 97 页

- **向上（下）兼容**：按某档机器编制的程序，不加修改就能运行于比它高（低）档的机器。
- **向前（后）兼容**：按某个时期投入市场的某种型号机器编制的程序，不加修改地就能运行于在它之前（后）投入市场的机器。**向后兼容是系列机的根本特征。**
- **兼容机**：由不同公司厂家生产的具有相同系统结构的计算机

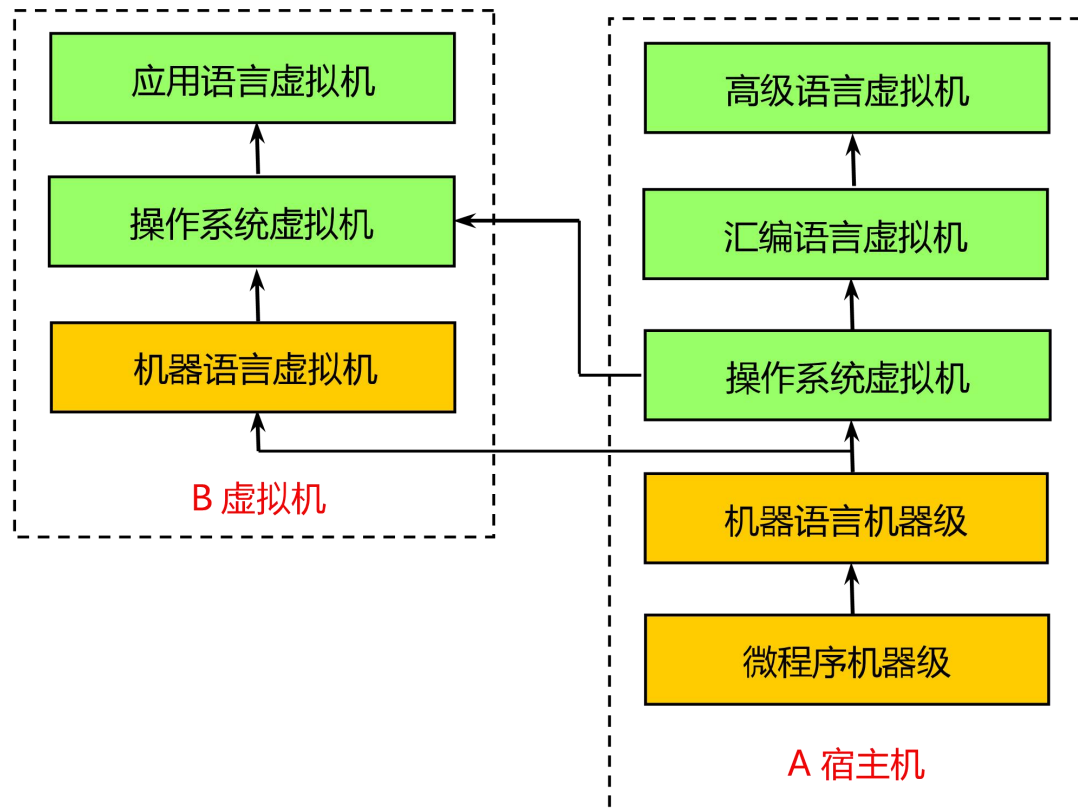


3. **模拟和仿真**：使软件能在具有不同系统结构的机器之间相互移植。

- 在一种系统结构上实现另一种系统结构。
- 从指令集的角度来看，就是要在一种机器上实现另一种机器的指令集

1.4.2 软件对系统结构的影响

第 98 页

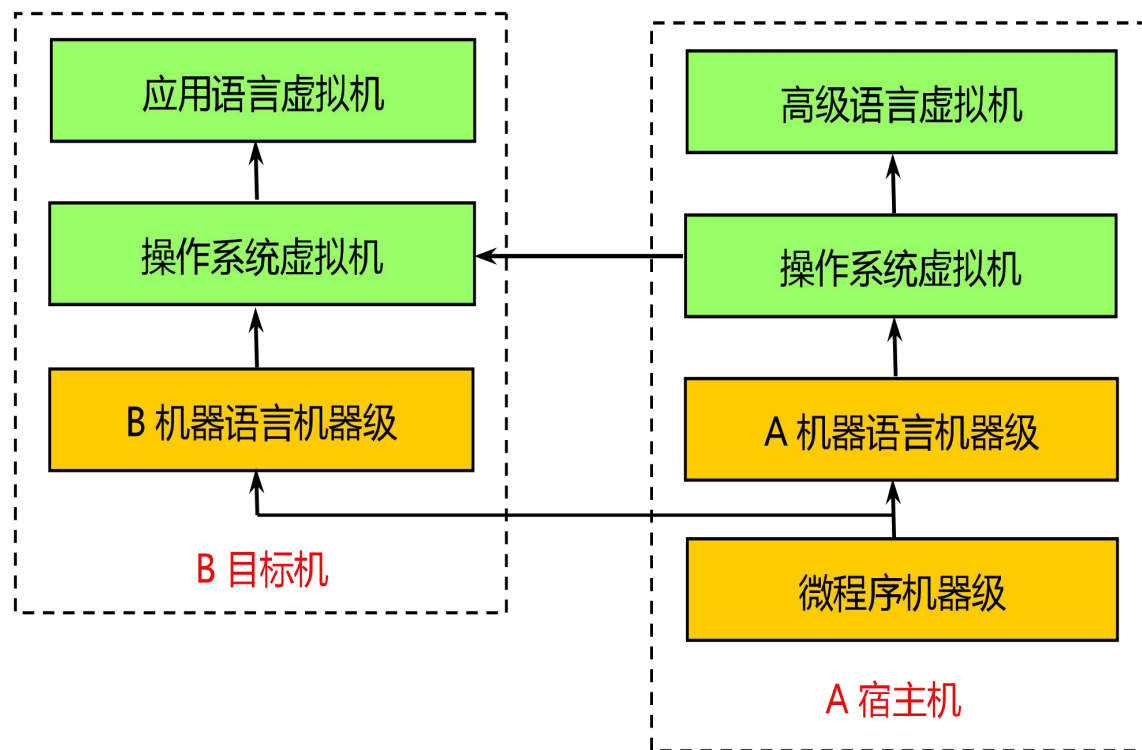


■ **模拟**：用软件的方法在一台现有的机器（称为**宿主机**）上实现另一台机器（称为**虚拟机**）的指令集。

- 通常用解释的方法来实现。
- 运行速度较慢，性能较差。

1.4.2 软件对系统结构的影响

第 99 页



■ **仿真**：用一台现有机器（**宿主机**）上的微程序去解释实现另一台机器（**目标机**）的指令集。

- 运行速度比模拟方法的快
- 仿真只能在系统结构差距不大的机器之间使用

1.4.3 器件发展对系统结构的影响

第 100 页

- 推动计算机系统结构不断发展的最活跃的因素
- 摩尔定律：集成电路芯片上所集成的晶体管数目每隔18个月就翻一番。
- 计算机的分代主要以器件作为划分标准。
 - 它们在器件、系统结构和软件技术等方面都有各自的特征。



1.4.3 器件发展对系统结构的影响

第一代 (1945-1954)	电子管和继电器	存储程序计算机、 程序控制I/O	机器语言和汇编语言	普林斯顿ISA、ENIAC、 IBM701
第二代 (1955-1964)	晶体管、磁芯、 印刷电路	浮点数据表示、寻址技术、中断、I/O 处理机	高级语言和编译、 批处理监控系统	Univac LARC、CDC1604、 IBM7030
第三代 (1965-1974)	SSI和MSI、多层 印刷电路、微程序	流水线、Cache、先行处理、系列计算机、CISC	多道程序和分时操作系统	IBM360/370、 CDC6600/7600、DEC PDP-8
第四代 (1974-1990)	LSI和VLSI、半导体存储器	向量处理、分布式存储器、RISC	并行与分布处理	Cray-1、IBM 3090、DEC VAX9000、Convax-1
第五代 (1991-)	高性能微处理器、高密度电路	指令级并行、SMP、MPP、MP、网络	大规模、可扩展并行与分布处理	SGI Cray T3E、IBM SP2、DEC AlphaServer8400

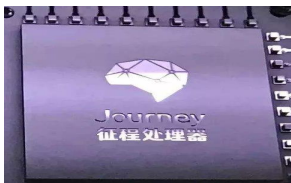
SMP：对称式共享存储器多处理机，MPP：大规模并行处理机，MP：多处理机

1.4.4 应用对系统结构的影响

第 102 页



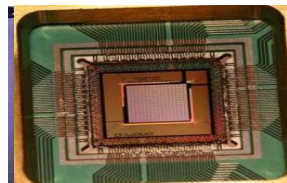
A P U — Accelerated Processing Unit, 加速处理器 (AMD公司)



B P U — Brain Processing Unit, 脑处理器 (地平线公司)



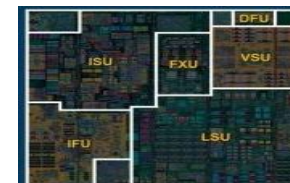
C P U — Central Processing Unit 中央处理器 (英特尔公司)



D P U — Dataflow Processing Unit 数据流处理器 (Wave Computing 公司)



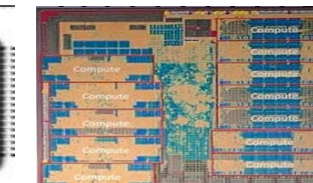
E P U — Energy Processing Unit, 能耗调控单元 (华硕公司)



F P U — Floating Processing Unit 浮点计算单元 (IBM公司)



G P U — Graphics Processing Unit, 图形处理器 (英伟达公司)



H P U — Holographics Processing Unit, 全息图像处理器 (微软公司)



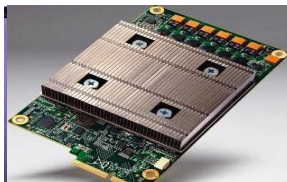
I P U — Intelligence Processing Unit, 智能处理器 (图核公司)



M P U — Microprocessor, 微处理器 (ARM公司)



N P U — Neural Network Processing Unit, 神经网络处理器



T P U — Tensor Processing Unit 张量处理器(谷歌公司)



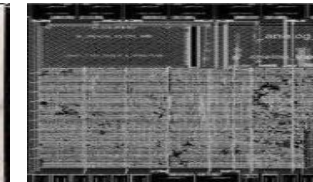
V P U — Vector Processing Unit, 矢量处理器 (英特尔公司)



W P U — Wearable Processing Unit, 可穿戴处理器



X P U — Xilinx Processing Unit, 云处理器 (Xilinx公司)



Z P U — Zylin Processing Unit, 一种开源处理器

CPU版摩尔定律：每过18天集成电路领域将多出一个xPU，直到26个字母被用完

1.4.4 应用对系统结构的影响

第 103 页

1. 不同的应用对计算机系统结构的设计提出了不同的要求。
2. 应用需求是促使计算机系统结构发展的最根本的动力。
3. 一些特殊领域：需要高性能的系统结构
 - 高结构化的数值计算
气象模型、流体动力学、有限元分析
 - 非结构化的数值计算
蒙特卡洛模拟、稀疏矩阵
 - 实时多因素问题
语音识别、图象处理、计算机视觉
 - 大存储容量和输入输出密集的问题
数据库系统、事务处理系统
 - 图形学和设计问题
计算机辅助设计
 - 人工智能
面向知识的系统、推理系统等

1.4.4 应用对系统结构的影响

第 104 页

■ 计算密集型应用 (computation-intensive)

- 科学与工程计算、数值模拟等
- 应用领域：石油、气象、CAD、核能、制药、环境监测分析、系统仿真等

■ 数据密集型应用 (data-intensive)

- 数字图书馆、数据仓库、数据挖掘、计算可视化等
- 应用领域：图书馆、银行、证券、税务、决策支持系统等

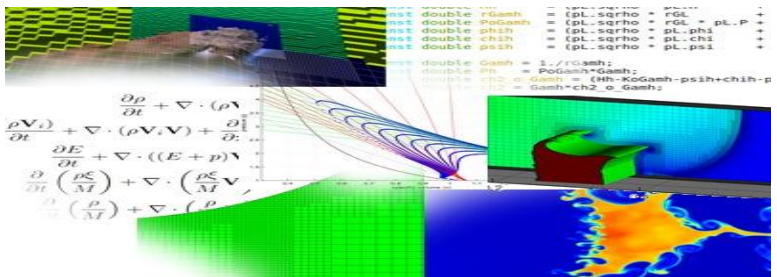
■ 网络密集型应用 (network-intensive)

- 协同工作、网格计算、遥控和远程诊断等
- 应用领域：网站、信息中心、搜索引擎、电信、流媒体等

■ 应用越来越复杂、网络与通信应用越来越多、云计算、大数据、智能系统等促进新型应用分类

1.4.4 应用对系统结构的影响

■**计算密集性**：计算密集型应用对每次从存储取出的数据进行大量算术运算，可以用计算访存比来量化



智能处理需求

- 数字处理器DSP、图像处理器GPU
- **并行计算单元多**
- 性能成本比、性能功耗比

嵌入式系统应用

— 计算密集型应用

计算密集需求



■**计算访存比**：对每次访存数据执行的计算操作个数（一般而言，计算密集型应用的平均计算访存比大于20）

计算密集型应用

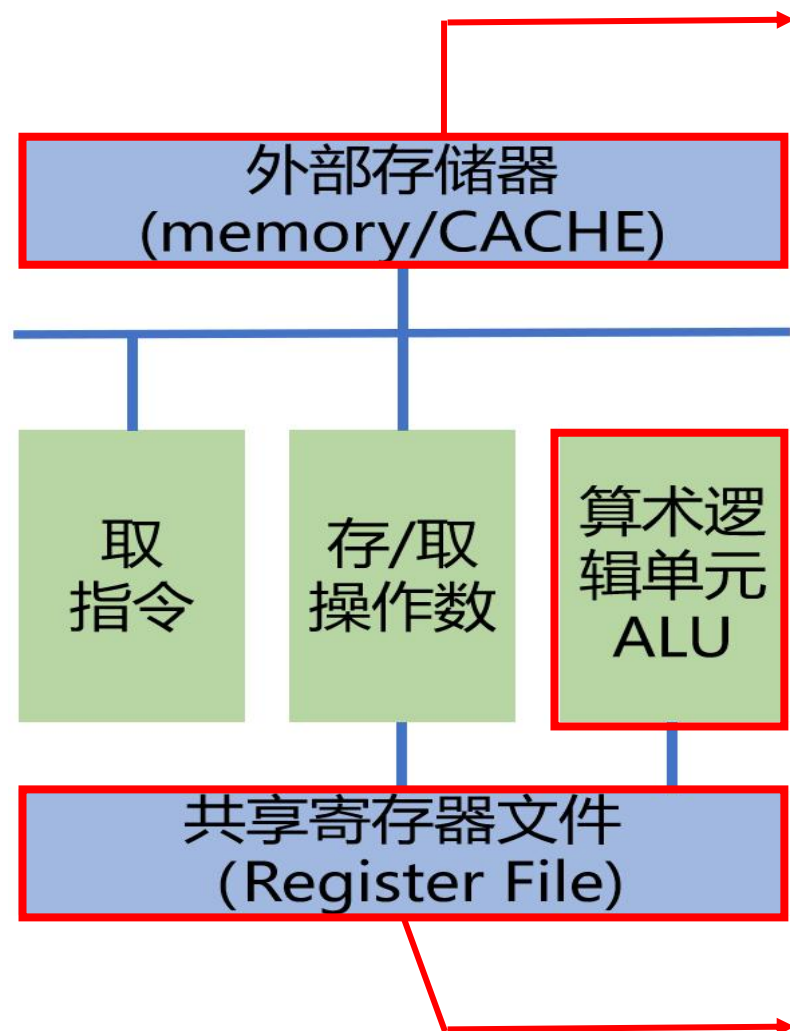
数据密集需求

网络 (IO) 密集需求

计算密集型应用	计算访存比
深度提取的卷积过滤与绝对值求和	473.3
视频的编码与解码 mpeg	57.9
矩阵的 QR 分解	155.3
Reed-solomn 解码	129.6
爆轰流体力学 Ygx2 (IAPCM Benchmarks)	76

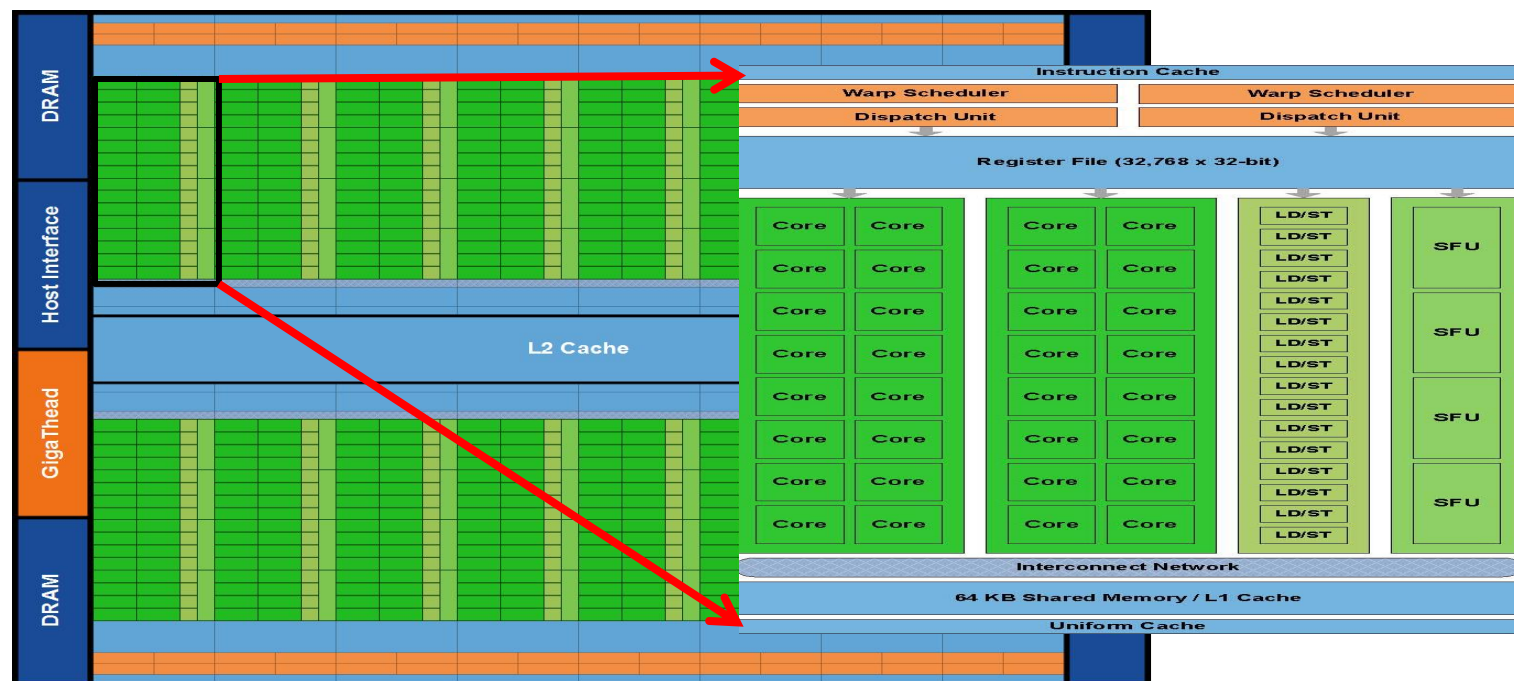
1.4.4 应用对系统结构的影响

第 106 页



架构相似、设计复杂

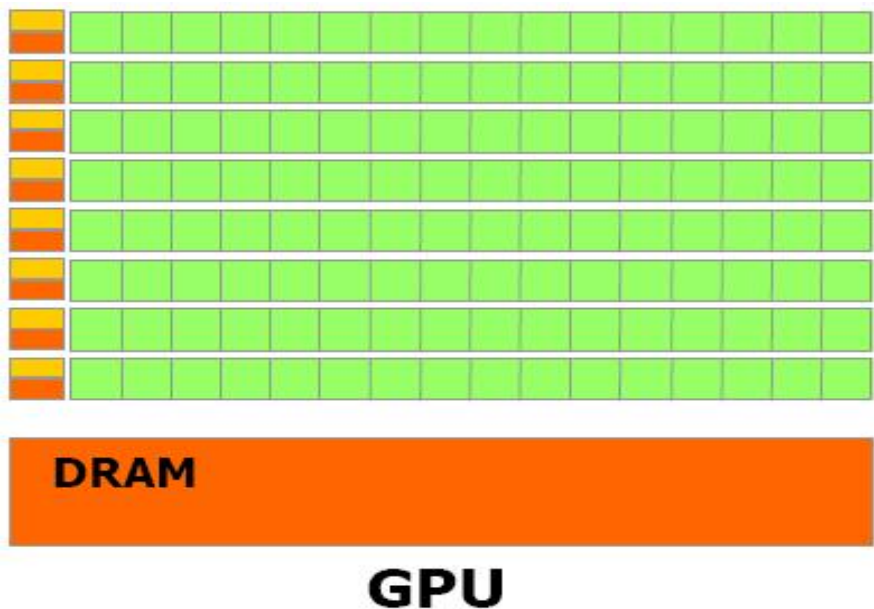
NVIDIA的GPU芯片——FERMI 结构



架构相似、设计复杂

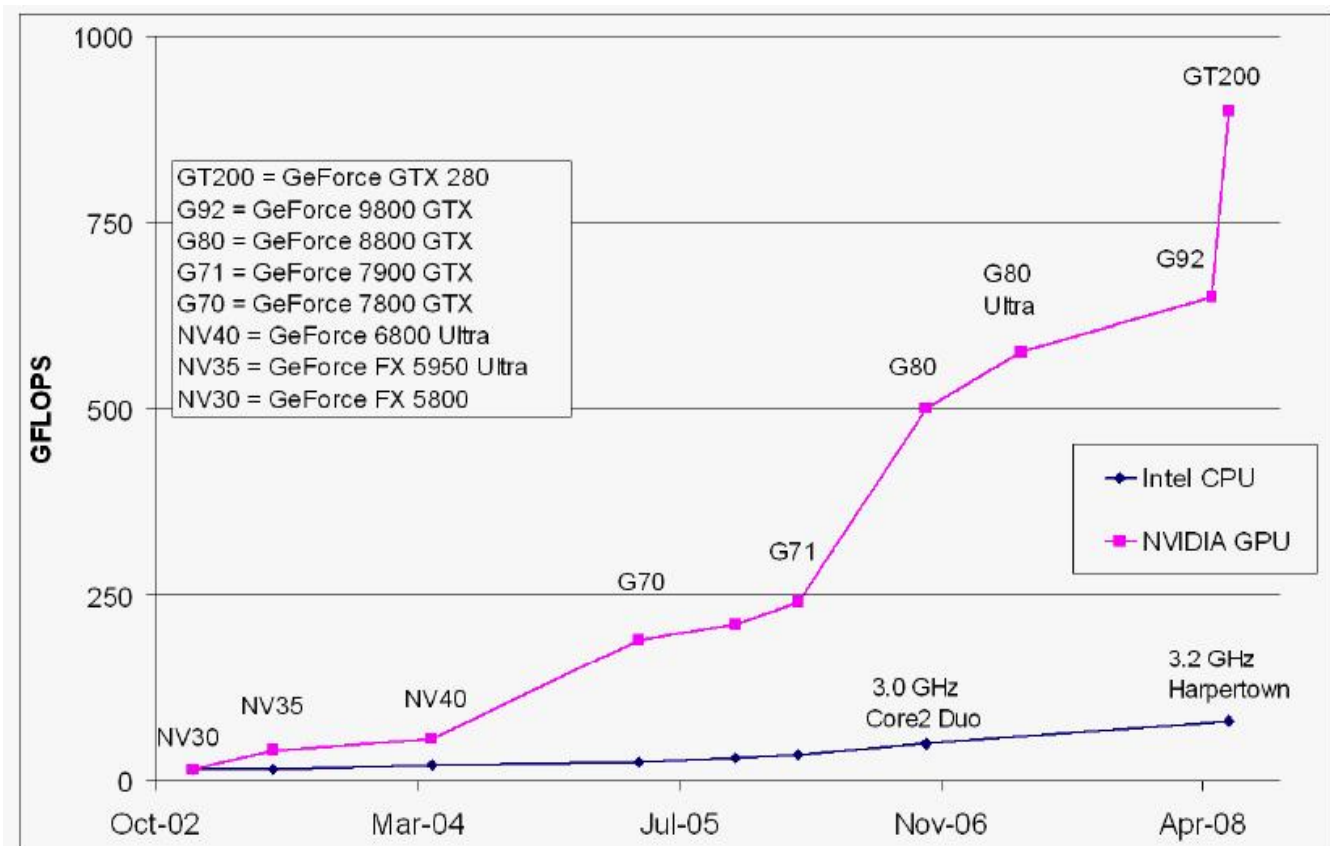
1.4.4 应用对系统结构的影响

第 107 页



GPU硬件架构突出了对CPU的优势：拥有高带宽的独立显存；浮点运算性能高；几何处理能力强；适合处理并行计算任务；适合进行重复计算；适合图像或视频处理任务；能够大幅度降低系统成本

GPU与通用处理器的性能比



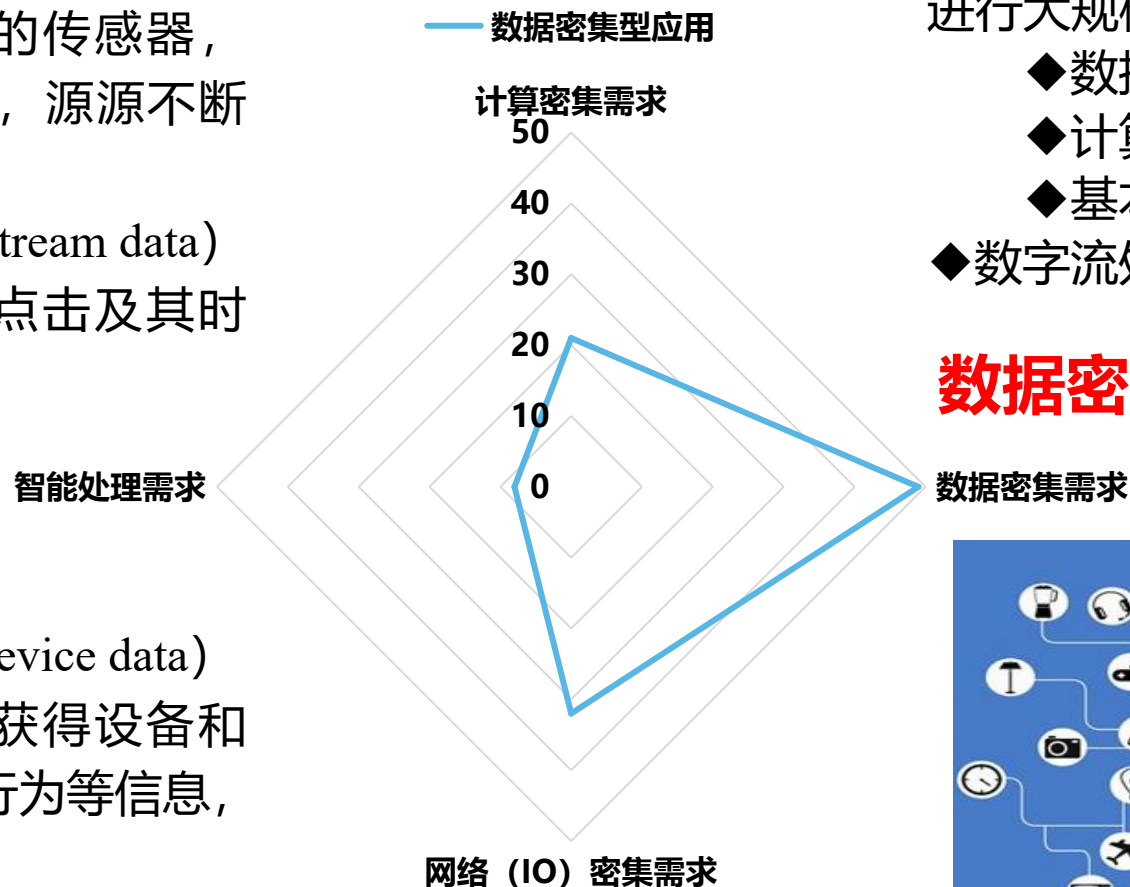
1.4.4 应用对系统结构的影响

第 108 页

■典型的数据来源

- ◆传感器数据 (sensor data)
 - 分布在不同位置上的传感器, 对所处环境进行感知, 源源不断地生成数据
- ◆网站点击流数据 (click stream data)
 - 用户在网上的每个点击及其时间被记录下来
- ◆移动设备数据 (mobile device data)
 - 通过移动电子设备获得设备和人员的位置、移动、行为等信息, 辅助决策, 如物联网

嵌入式系统应用



■数据密集型应用是以数据为中心, 系统负责获取、维护持续改变的数据集进行大规模的计算和处理。

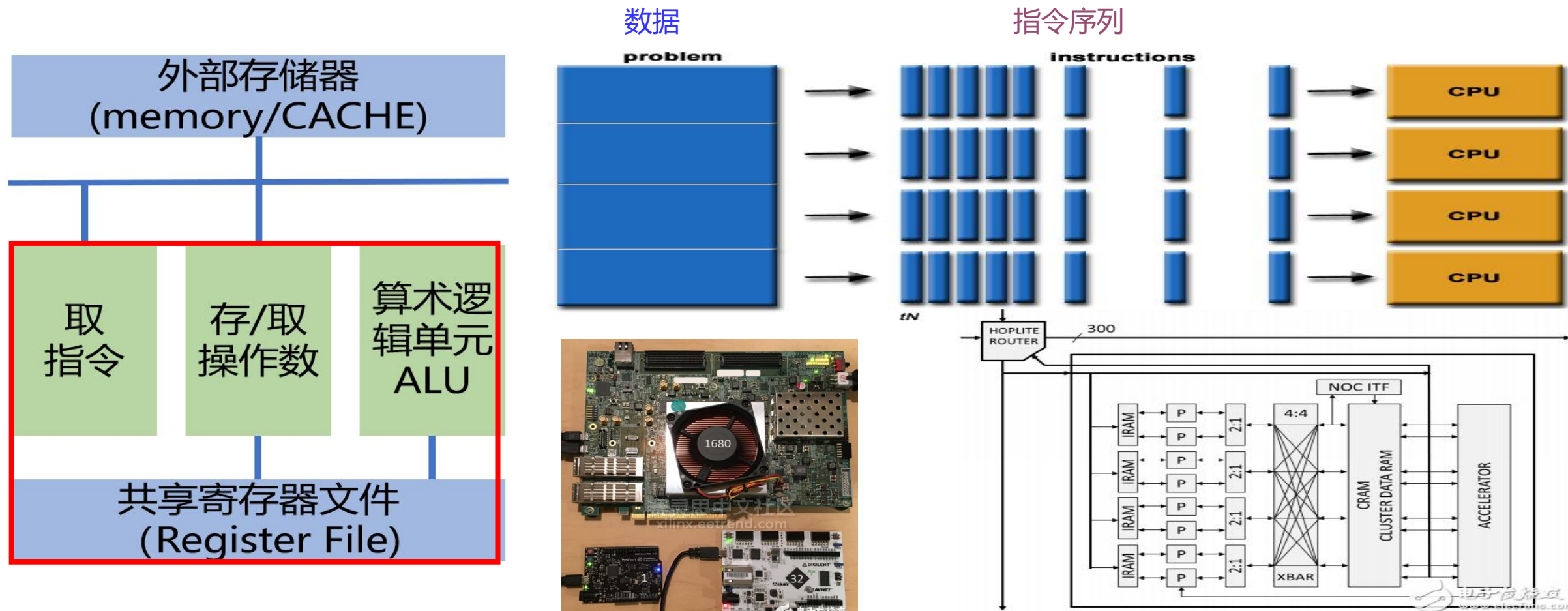
- ◆数据并行
- ◆计算并行
- ◆基本思想
- ◆数字流处理器、FPGA、多核

数据密集型应用



1.4.4 应用对系统结构的影响

第 109 页



GRVI Phalanx的设计由210个处理器集群组成，每个集群中包含了8个开源的RISC-V处理器核，128KB的多端口RAM，以及一个300位的Hoplite NOC路由

1.4.4 应用对系统结构的影响

第 110 页

■网络应用：有线网络+无线网络

- ◆2021年，中国网购人群8.42亿，占网民的81.6%
- ◆英法德意四国总人口约2.7亿
- ◆2021年，中国网民10.32亿，智能手机用户8.5亿
- ◆链路带宽迅速提高
 - 增长速度大于CPU增长速度
- ◆新的业务大量涌现
- ◆业务流量持续增长
 - 每12个月翻一番
- ◆包处理越来越复杂
 - 安全、数据包检查、.....

2013-2020年中国网购交易额PC端和移动端占比



1.4.4 应用对系统结构的影响

第 111 页

■新型网络处理器 (network processor)

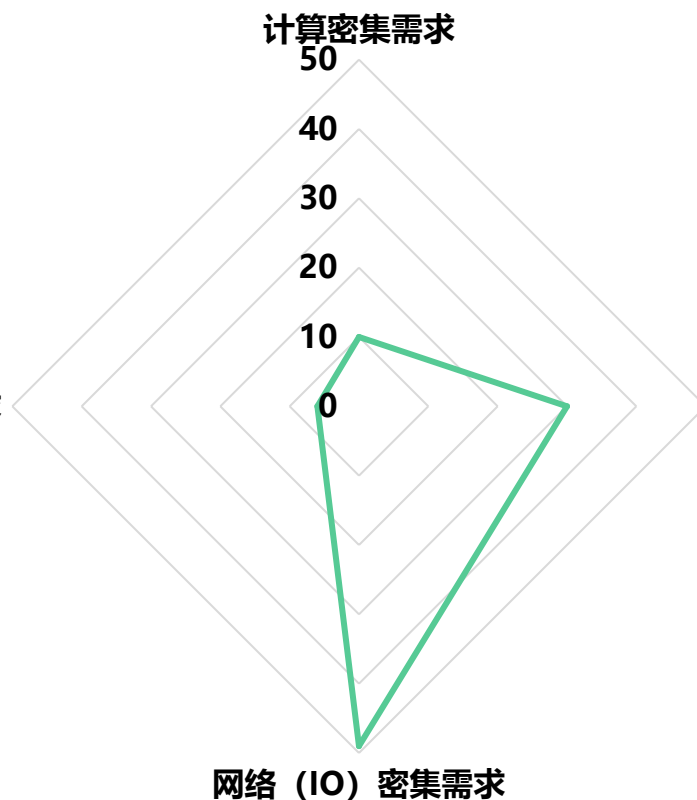
- ◆实现网络控制、数据传输和系统管理
- ◆负责网络层、传输层、协议层、应用层等多层管理
- ◆支持嵌入式操作系统和高级语言 (C++、JAVA) 编程

■典型的网络处理器

- ◆Broadcom (MIPS架构)、RMI (MIPS架构)、CISCO、Freescale (PowerPC架构)、Tilera

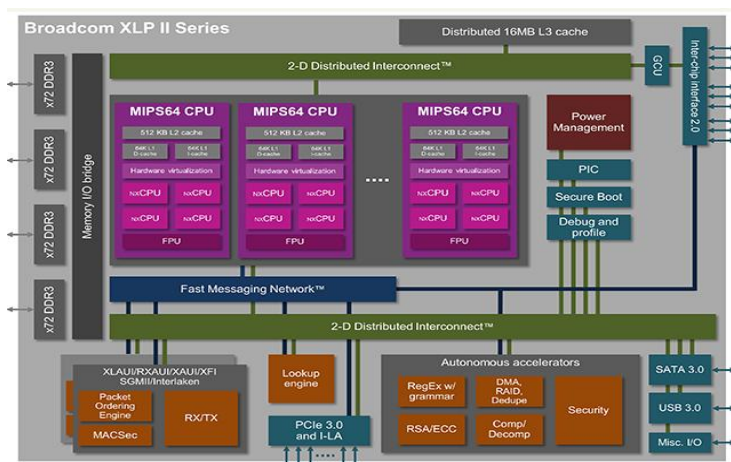
嵌入式系统应用

— 网络密集型应用



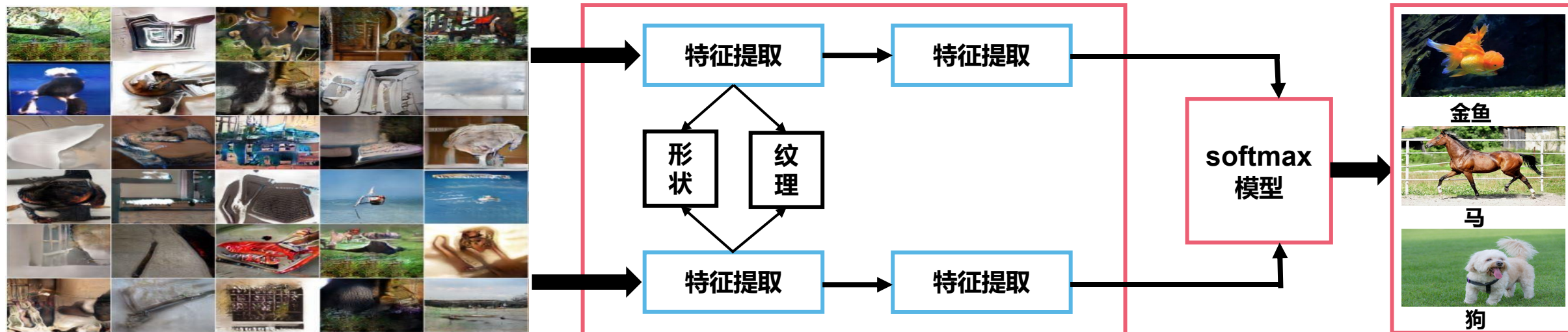
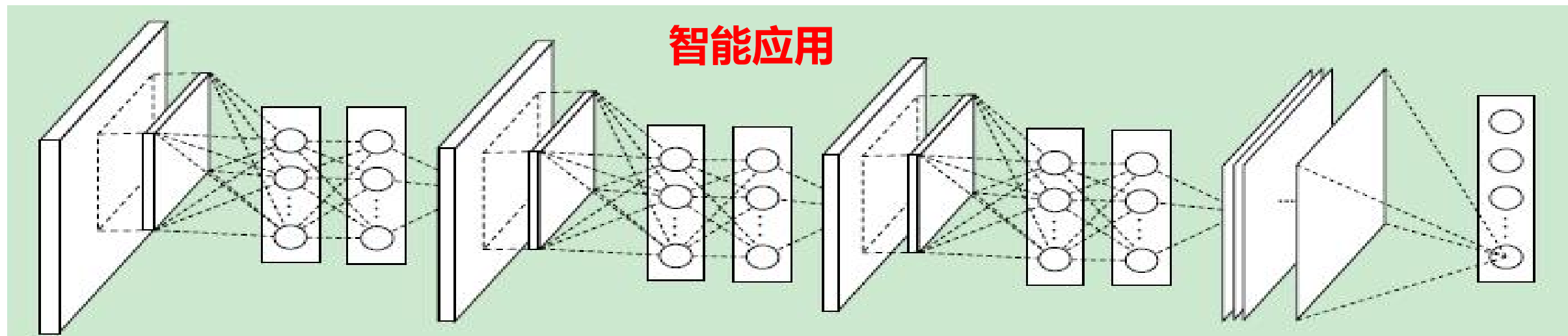
Broadcom XLP900 处理器有超过80个核连接到快速的信息网络和分布式的连接, 支持硬件虚拟交叉一个阵列达到640个核。

网络密集型应用



1.4.4 应用对系统结构的影响

第 112 页

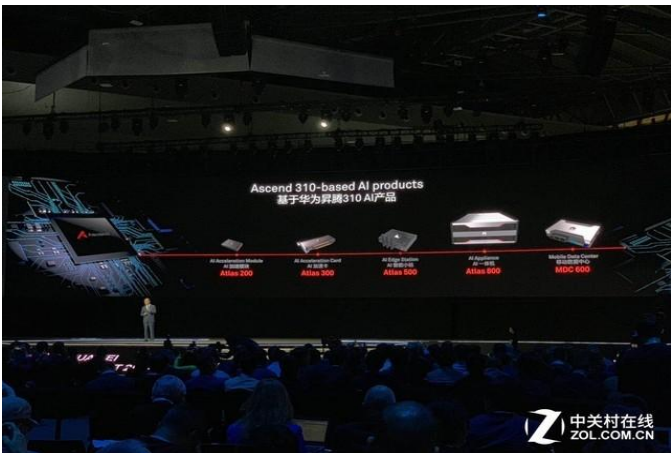


1.4.4 应用对系统结构的影响

2018年10月，代号为910和310的昇腾AI芯片系列产品推出。



- 昇腾910处理器计算密度较大，采用了7nm先进工艺制程，最大功耗为350W，FP16算力可以达到256TFLOPS，单芯片计算密度领先全球。INT8算力可以达到512TOPS，同时支持128通道全高清视频解码（H.264/H.265）。
- 昇腾910处理器主要应用于云端，可以为深度学习的训练算法提供强大算力。



浮点运算能力对比		
公司	产品	性能
谷歌	Cloud TPU v3	420 TFLOPS
华为	昇腾910	256 TFLOPS
英伟达	Tesla V100 GPU	125 TFLOPS

1.5 计算机系统结构中并行性的发展

1.5.1 并行性的概念

第 115 页

1. 并行性：计算机系统在同一时刻或者同一时间间隔内进行多种运算或操作。只要在时间上相互重叠，就存在并行性。

- 同时性：两个或两个以上的事件在同一时刻发生。
- 并发性：两个或两个以上的事件在同一时间间隔内发生。

2. 从处理数据的角度来看，并行性等级从低到高可分为：

- 字串位串：每次只对一个字的一位进行处理。最基本的串行处理方式，不存在并行性。
- 字串位并：同时对一个字的全部位进行处理，不同字之间是串行的。**开始出现并行性。**
- 字并位串：同时对许多字的同一位（称为位片）进行处理。**具有较高的并行性。**
- 全并行：同时对许多字的全部位或部分位进行处理。**最高一级的并行。**

1.5.1 并行性的概念

第 116 页

3. 从执行程序的角度来看，并行性等级从低到高可分为：

- 
- 指令内部并行：单条指令中各微操作之间的并行。
 - 指令级并行：并行执行两条或两条以上的指令。
 - 线程级并行：并行执行两个或两个以上的线程。通常是以一个进程内派生的多个线程为调度单位。
 - 任务级或过程级并行：并行执行两个或两个以上的过程或任务（程序段），以子程序或进程为调度单元。
 - 作业或程序级并行：并行执行两个或两个以上的作业或程序。

三种途径：

1. 时间重叠

- 引入时间因素，让多个处理过程在时间上相互错开，轮流重叠地使用同一套硬件设备的各个部分，以加快硬件周转而赢得速度。

2. 资源重复

- 引入空间因素，以数量取胜。通过重复设置硬件资源，大幅度地提高计算机系统的性能。

3. 资源共享

- 这是一种软件方法，它使多个任务按一定时间顺序轮流使用同一套硬件设备。

1.5.3 单机系统中并行性的发展

第 118 页

1. 在发展高性能单处理机过程中，起主导作用的是时间重叠原理；实现时间重叠的基础：部件功能专用化
 - 把一件工作按功能分割为若干相互联系的部分；把每一部分指定给专门的部件完成；然后按时间重叠原理把各部分的执行过程在时间上重叠起来，使所有部件依次分工完成一组同样的工作。
2. 在单处理机中，资源重复原理的运用也已经十分普遍。
 - 多体存储器、多操作部件等
 - 指令级并行：通用部件被分解成若干个专用部件，如加法部件、乘法部件、除法部件、逻辑运算部件等，而且同一种部件也可以重复设置多个。只要指令所需的操作部件空闲，就可以开始执行这条指令（如果操作数已准备好的话）。

3. 阵列处理机（并行处理机）：

- 设置许多相同的处理单元，让它们在同一个控制器的指挥下，按照同一条指令的要求，对向量或数组的各元素同时进行同一操作，就形成了阵列处理机。

4. 分时系统：

- 在单处理机中，资源共享的概念实质上是用单处理机模拟多处理机的功能，形成所谓虚拟机的概念。

1.5.4 多机系统中并行性的发展

第 120 页

1. 多机系统遵循时间重叠、资源重复、资源共享原理，发展为3种不同的多处理机：

- ① 同构型多处理机
- ② 异构型多处理机
- ③ 分布式系统

2. 耦合度：反映多机系统中各机器之间物理连接的紧密程度和交互作用能力的强弱。

- 紧密耦合系统（直接耦合系统）：在这种系统中，计算机之间的物理连接的频带较高，一般是通过总线或高速开关互连，可以共享主存。
- 松散耦合系统（间接耦合系统）：一般是通过通道或通信线路实现计算机之间的互连，可以共享外存设备（磁盘、磁带等）。

1.5.5 并行机的发展变化

第 122 页

并行机的发展可分为5个阶段。

1. 并行机的萌芽阶段（1964年～1975年）

- 20世纪60年代初期：CDC6600：非对称的共享存储结构，中央处理机采用了双CPU，并连接了多个外部处理器。
- 60年代后期，一个重要的突破：在处理器中使用流水线和重复设置功能单元，所获得的性能提高是明显的，并比单纯地提高时钟频率来提高性能更有效。
- 在1972年，Illinois大学和Burroughs公司联合研制Iliac IV SIMD计算机（64个处理单元构成的）在1975年 Iliac IV系统（16个处理单元构成）

1.5.5 并行机的发展变化

第 123 页

2. 向量机的发展和鼎盛阶段（1976年～1990年）

- 1976年，Cray公司推出了第一台向量计算机Cray-1。
- 在随后的10年中，不断地推出新的向量计算机。包括：CDC的Cyber205、Fujitsu的VP1000/VP2000、NEC的SX1/SX2以、我国的YH-1等。
- 向量计算机的发展呈两大趋势：
 - 提高单处理器的速度
 - 研制多处理器系统

1.5.5 并行机的发展变化

第 124 页

3. MPP（大规模并行处理机）出现和蓬勃发展阶段（1990年～1995年）

- 早期的MPP：TC2000（1989年）、Touchstone Delta、Intel Paragon（1992年）、KSR1、Cray T3D（1993年）、IBM SP2（1994年）和我国的曙光-1000（1995年）等。（分布存储的MIMD计算机）
- MPP的高端机器：1996年，Intel公司的ASCI Red和1997年SGI Cray公司的T3E900（万亿次高性能并行计算机）
- 90年代的中期，在中、低档市场上，SMP以其更优的性能/价格比代替了MPP。

4. 各种体系结构并存阶段（1995年～2000年）

- 从1995年以后，PVP（并行向量处理机）、MPP、SMP、DSM（分布式共享存储多处理机）、COW等各种体系结构进入并存发展的阶段。
- MPP系统在全世界前500强最快的计算机中的占有量继续稳固上升，其性能也得到了进一步的提高。如：ASCI Red的理论峰值速度已达到了1Tflop/s，SX4和VPP700等的理论峰值速度也都达到了1Tflop/s。

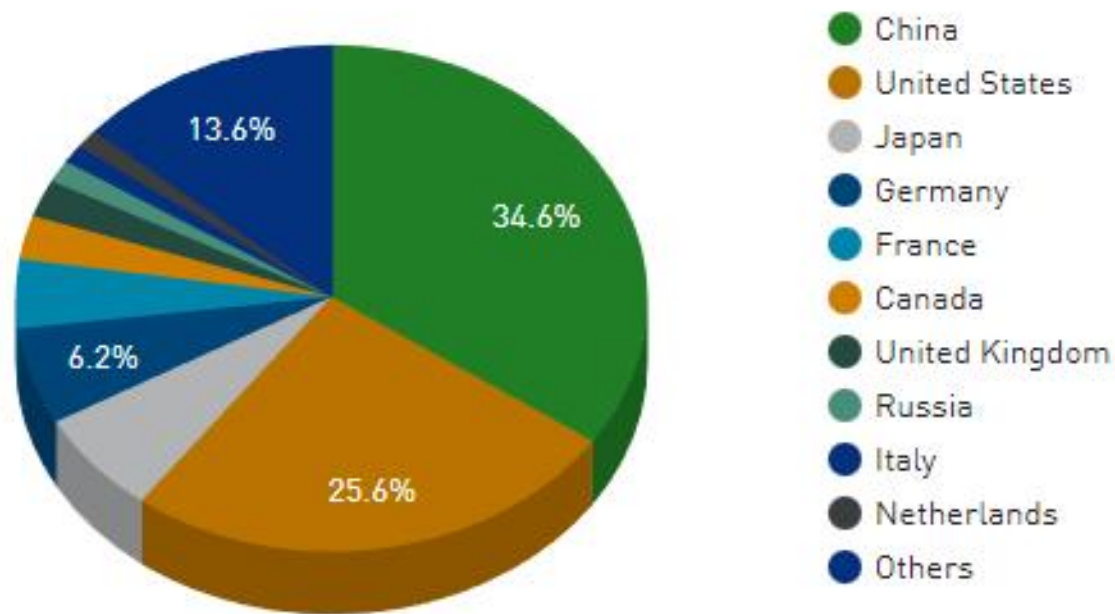
5. 机群蓬勃发展阶段（2000年以后）

- 机群系统：将一群工作站或高档微机用某种结构的互连网络互连起来，充分利用其中各计算机的资源，统一调度、协调处理，以达到很高的峰值性能，并实现高效的并行计算。
- 1997年6月才有第一台机群结构的计算机进入Top500排名
- 2003年11月，这一数字已达到208台，机群首次成为Top500排名中比例最高的结构。截至2008年6月，机群已经连续10期位居榜首，其数量已经达到400，占80%。

1.5.5 并行机的发展变化

第 127 页

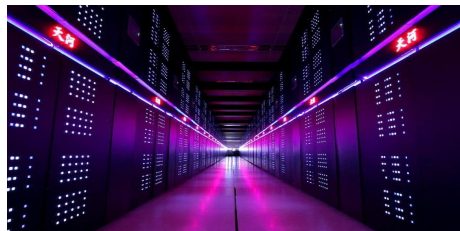
Countries System Share



引自www.top500.org



神威·太湖之光 (2016)
千万核
异构众核处理器SW26010
OpenACC*, Athread + MPI



天河2A (2017)
498万核
2xCPU + 2xMatrix2000
OpenMP, OpenCL + MPI



先导1号 (2018)
6550节点
海光CPU + 4xDCU
HCC, HIP, OpenCL + MPI

过去十年，我国在顶尖超算系统研制和超算系统部署数量都处于国际领先行列

1. 计算机系统结构的基本概念——层次结构，翻译与解释，组成与实现，系列机与Flynn分类法、冯氏分类法和Handler分类法等
2. 计算机系统的设计——经常性事件原则，Amdahl定律，CPU性能公式，程序局部性原理以及由项向下等设计流程
3. 计算机系统的性能评测——执行时间和吞吐率，基准测试程序，总执行时间，算术平均时间，调和平均时间，加权执行时间及标准化执行时间等
4. 计算机系统结构的发展——冯·诺依曼结构及其改进，器件、软件与应用对系统结构的影响
5. 计算机系统结构中并行性的发展——并行性概念，提高并行性的技术途径以及并行性发展阶段总结

选择题

第 130 页

- 5、不同系列的机器之间，实现可移植性的途径不包括（ **B** ）
- A. 采用统一高级语言 B. 采用统一汇编语言 C. 模拟 D. 仿真
- 6、利用时间重叠原理实现并行处理的是（ **A** ）
- A. 流水处理机 B. 多处理机 C. 阵列处理机 D. 机群系统
- 7、多处理机实现的并行主要是（ **B** ）
- A. 指令级并行 B. 任务级并行 C. 操作级并行 D. 操作步骤的并行
- 8、从计算机系统结构来看，机器语言程序员看见的机器属性是（ **C** ）
- A. 计算机软件所要完成的功能 B. 计算机硬件的全部组成
- C. 编程要用到的硬件组织 D. 计算机各部件的硬件实现

应用题

第 131 页

1、某台主频为400MHz的计算机执行标准测试程序，程序中指令类型，执行数量和平均周期数如下表所示，求该计算机的有效CPI、MIPS和程序执行时间。

指令类型	指令执行数量	平均时钟周期数
整数	4000	1
数据传递	8000	2
浮点	6000	4
分支	2000	2

解 $CPI = (4000 \times 1 + 8000 \times 2 + 6000 \times 4 + 2000 \times 4) / 20000 = 2.6$

$MIPS = f / CPI = 153.8 \text{ MIPS}$

程序执行时间 = $(4000 \times 1 + 8000 \times 2 + 6000 \times 4 + 2000 \times 4) / 400 = 130s$

应用题

第 132 页

2、假设处理机的时钟频率为40MHz，运行的测试程序共有200000条指令，由4类指令组成，已知各类指令的CPI和各类指令条数的比例如下表所示，计算运行该程序的CPI和MIPS。

指令类型	CPI	指令条数比例
算术逻辑运算指令	1	60%
Cache命中的load/store指令	2	18%
转移类指令	4	12%
Cache不命中的load/store指令	8	10%

解
$$CPI = \sum(CPI_i \times IC_i / IC) = 1 \times 0.6 + 2 \times 0.18 + 4 \times 0.12 + 8 \times 0.1 = 2.24$$

$$MIPS = \frac{\text{时钟频率}}{CPI \times 10^6} = \frac{40 \times 10^6}{2.24 \times 10^6} = 17.86 \text{ MIPS}$$

3、已知4个程序在3台计算机上的执行时间如下表所示，假设4个程序都执行1000000000条指令，计算这3台计算机中每台机器上每个程序的MIPS速率，分别计算它们的算术平均值、几何平均值和调和平均值。

程序	执行时间		
	计算机A	计算机B	计算机C
程序1	1	10	20
程序2	1000	100	20
程序3	500	1000	50
程序4	100	800	100

先计算MIPS值，再计算算术平均值、几何平均值和调和平均值

算术平均

$$A_m = \frac{1}{n} \sum_{i=1}^n T_i$$

调和平均

$$H_m = \frac{n}{\sum_{i=1}^n \frac{1}{T_i}}$$

几何平均

$$G_m = \sqrt[n]{\prod_{i=1}^n ETR_i}$$

应用题

3、已知4个程序在3台计算机上的执行时间如下表所示，假设4个程序都执行1000000000条指令，计算这3台计算机中每台机器上每个程序的MIPS速率，分别计算它们的算术平均值、几何平均值和调和平均值。

解

程序	MIPS		
	计算机A	计算机B	计算机C
程序1	100	10	5
程序2	0.1	1	5
程序3	0.2	0.1	2
程序4	1	0.125	1

程序	执行时间		
	计算机A	计算机B	计算机C
算术平均值	25.3	2.81	3.25
几何平均值	1.19	0.59	2.66
调和平均值	0.25	0.2	2.1

4、计算机系统有3个部件可以改进，这3个部件的加速比如下：

部件加速比 $S_1 = 30$ ；部件加速比 $S_2 = 20$ ；部件加速比 $S_3 = 10$

(1) 如果部件1和部件2的可改进比例为30%，那么当部件3的可改进比例为多少时，系统的加速比才可以达到10？

(2) 如果3个部件的可改进比例为30%、30%和20%，3个部件同时改进，则改进后系统中不可改进部分的执行时间在总执行时间中占的比例是多少？

解：(1) 多个部分可改进的情况下，Amdahl定理可扩展为

$$S_n = \frac{T_0}{T_n} = \frac{1}{(1 - F_e) + \frac{F_e}{S_e}} \quad \longrightarrow \quad S_n = \frac{1}{(1 - \sum F_i) + \sum \frac{F_i}{S_i}}$$

已经 $S_1 = 30$, $S_2 = 20$, $S_3 = 10$, $S_n = 10$, $F_1 = 0.3$, $F_2 = 0.3$, 求 F_3 , 则

$$10 = \frac{1}{(1 - 0.3 - 0.3 - F_3) + (0.3/30 + 0.3/20 + F_3/10)} \quad \longrightarrow \quad F_3 = 0.36, \text{ 可改进比例 } 36\%$$

4、计算机系统有3个部件可以改进，这3个部件的加速比如下：

部件加速比 $S_1 = 30$ ；部件加速比 $S_2 = 20$ ；部件加速比 $S_3 = 10$

(2) 如果3个部件的可改进比例为30%、30%和20%，3个部件同时改进，则改进后系统中不可改进部分的执行时间在总执行时间中占的比例是多少？

解：(2) 设改进前系统的总执行时间为 T ，则不可改进部分的执行时间 S_e

$$S_e = (1 - 0.3 - 0.3 - 0.2)T = 0.2T$$

改进后系统的总执行时间为 T'_n 为

$$T'_n = 0.2T + (0.3/30 + 0.3/20 + 0.2/10)T = 0.245T$$

改进后系统中不可改进部分的执行时间在总执行时间中占的比例是

$$\frac{0.2T}{0.245T} = 0.82 \quad (82\%)$$

5、假设某应用程序中有4类操作，通过改进，各操作获得不同的性能提高。具体数据如下表所示：

操作类型	程序中的数量 (百万条指令)	改进前的执行时间 (周期)	改进后的执行时间 (周期)
操作1	10	2	1
操作2	30	20	15
操作3	35	10	3
操作4	15	4	1

- (1) 改进后，各类操作的加速比分别是多少？
- (2) 各类操作单独改进后，程序获得的加速比分别是多少？
- (3) 4类操作均改进后，整个程序的加速比是多少？

5、假设某应用程序中有4类操作，通过改进，各操作获得不同的性能提高。具体数据如下表所示：

操作类型	程序中的数量 (百万条指令)	改进前的执行时间 (周期)	改进后的执行时间 (周期)
操作1	10	2	1
操作2	30	20	15
操作3	35	10	3
操作4	15	4	1

$$S_n = \frac{1}{(1 - Fe) + \frac{Fe}{Se}}$$

(1) 改进后，各类操作的加速比 (2) 各类操作单独改进后程序获得的加速比

操作类型	各类操作的指令条数在程序中所占的比例 F_i	各类操作的加速比 S_i	各类操作单独改进后，程序获得的加速比
操作1	10/90=11.1%	2/1=2	1.06
操作2	30/90=33.3%	20/15=1.33	1.09
操作3	35/90=38.9%	10/3=3.33	1.37
操作4	15/90=16.7%	4/1=4	1.14

应用题

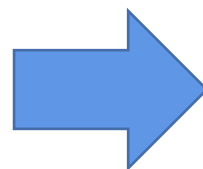
第 139 页

5、假设某应用程序中有4类操作，通过改进，各操作获得不同的性能提高。具体数据如下表所示：

操作类型	各类操作的指令条数在程序中所占的比例 F_i	各类操作的加速比 S_i	各类操作单独改进后，程序获得的加速比
操作1	10/90=11.1%	2/1=2	1.06
操作2	30/90=33.3%	20/15=1.33	1.09
操作3	35/90=38.9%	10/3=3.33	1.37
操作4	15/90=16.7%	4/1=4	1.14

(3) 4类操作均改进后，整个程序的加速比是多少？

$$S_n = \frac{1}{(1 - \sum F_i) + \sum \frac{F_i}{S_i}}$$



$$S_n \approx 2.16$$

6、假设我们考虑条件分支指令的两种不同设计方法：

(1) CPU_A：通过比较指令设置条件码，然后测试条件码进行分支。

(2) CPU_B：在分支指令中包括比较过程。

在这种两种CPU中，条件分支指令都占用2个时间周期，而所有其他指令占用1个时钟周期。对于CPU_A，执行的指令中分支指令占20%；由于每条分支指令之间都需要有比较指令，因此比较指令也占20%。由于CPU_A在分支时不需要比较，因此CPU_B的时钟周期时间是CPU_A的1.25倍。问：哪一个CPU更快？如果CPU_B的时钟周期时间只是CPU_A的1.1倍，哪一个CPU更快？

解：参见课件。

7、若在整个测试程序的执行时间中，求浮点数平方根FPSQR的操作占10%。现有两种改进方案：

(1) 采用FPSQR硬件，使其速度加快到10倍；

(2) 使所有浮点数指令FP速度加快到4倍，并设FP指令占整个程序执行时间的40%，试比较两种方案的优劣。

解：(1) $F_e=0.1$, $S_e=10$, 则

$$S_n = \frac{1}{(1-0.1) + \frac{0.1}{10}} \approx 1.1$$

(2) $F_e=0.4$, $S_e=4$, 则

$$S_n = \frac{1}{(1-0.4) + \frac{0.4}{4}} \approx 1.43$$

比较结果得：方案 (2) 更优

8、假设浮点数指令FP的比例为30%，其中浮点数平方根FPSQR占全部指令的比例为4%，FP操作的CPI为5，FPSQR操作的CPI为20，其他指令的平均CPI为1.25。现有两种改进方案，第一种是把FPSQR操作的CPI减至3，第二种是把所有的FP操作为CPI减至3，试比较两种方案对系统性能的提高程度。

解：没有改进之前，每条指令的平均时钟周期CPI为：

$$CPI = \sum_{i=1}^n (CPI_i \times \frac{IC_i}{IC}) = (5 \times 30\%) + (1.25 \times 70\%) = 2.38$$

(1) 采用第一种方案：FPSQR操作的CPI由 $CPI_{FPSQR}=20$ 减至 $CPI'_{FPSQR}=3$ ，则整个系统的指令平均时钟周期数为：

$$CPI_1 = CPI - (CPI_{FPSQR} - CPI'_{FPSQR}) \times 4\% = 2.38 - (20 - 3) \times 4\% = 1.7$$

更优

(2) 采用第二种方案：所有FP操作的CPI由 $CPI_{FP}=5$ 减至 $CPI'_{FP}=3$ ，则整个系统的指令平均时钟周期数为：

$$CPI_2 = CPI - (CPI_{FP} - CPI'_{FP}) \times 30\% = 2.38 - (5 - 3) \times 30\% = 1.78$$

第2章 指令系统的设计